Hardware Design Lab

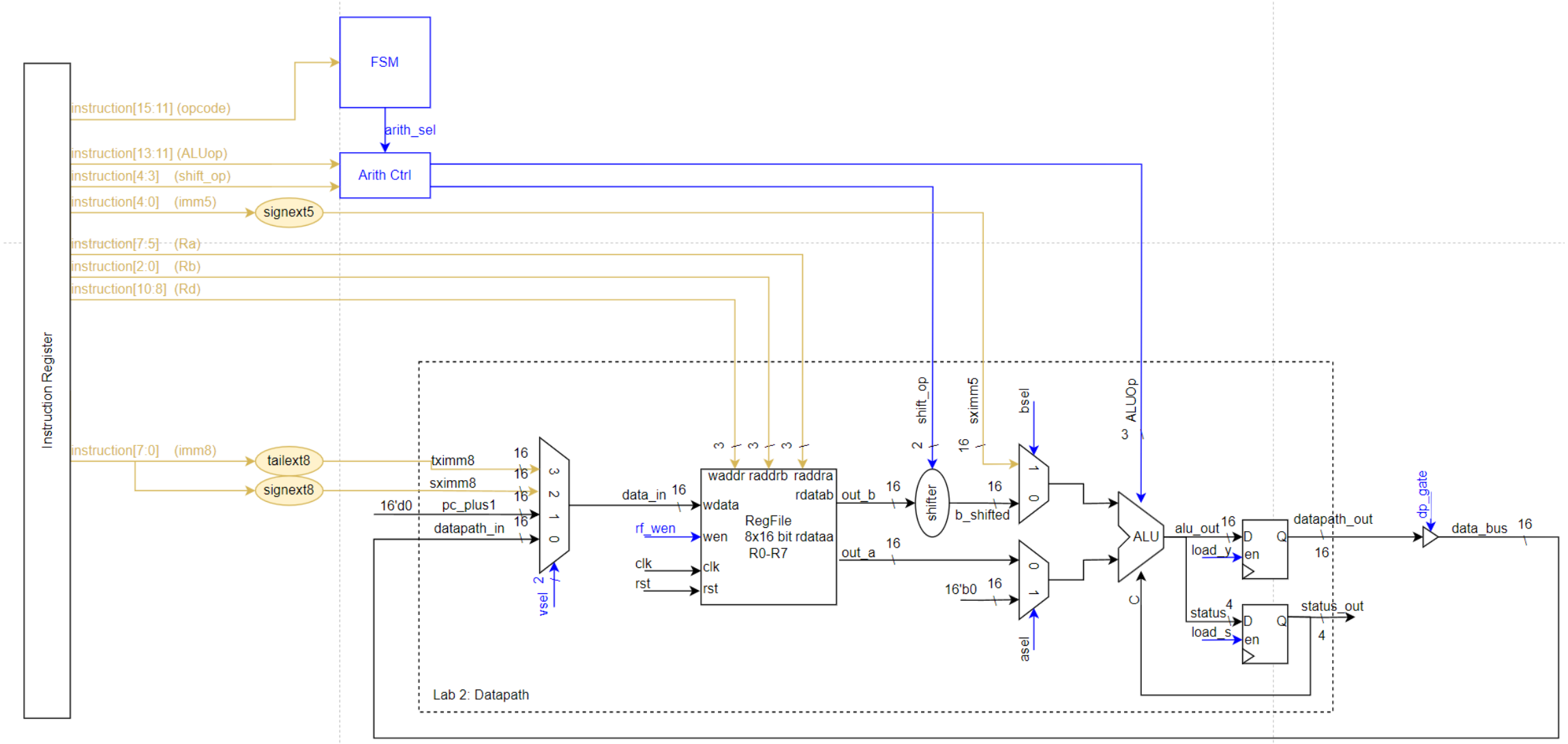
Task 4: Complete CPU

Instructor: Dr. Haiyu Mao
TA: Zihao Pu
Ali Alsarraf
Stephen Johannesson

Mar 5, 2026

Gratefully acknowledge:
Prof. Onur Mutlu (ETH)
Dr. Yair Linn (TRIUMF Canada)
Prof. Tor Aamodt (UBC)

Lab 3 Recap: What we have accomplished

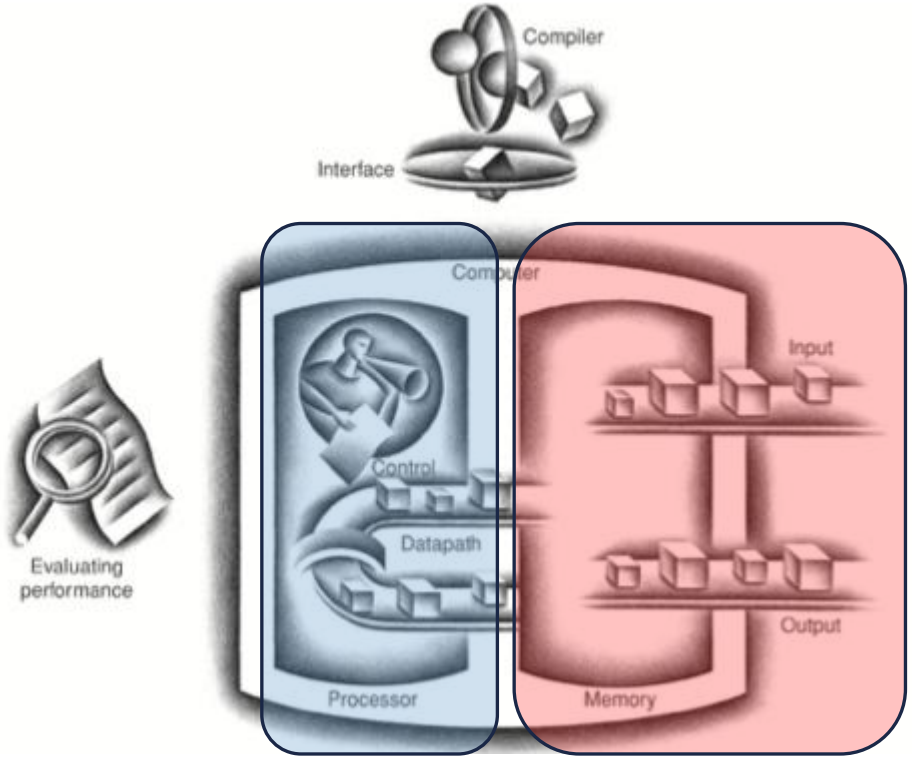


Lab 3 Recap: Supporting Instructions

Mnemonic	Type	OpCode[15:11]		DST [10:8]	SRC1 [7:5]	SHIFT [4:3]	SRC2 [2:0]	Flags	Description & Operation
ARITHMETIC		Op[15:14]	ALUop[13:11]	Rd[10:8]	Ra[7:5]	shiftop[4:3]	Rb[2:0]		(Shifter applies to Rs2)
ADD	R	00	000	Rd	Ra	Shift	Rb	Z, N, C, V	Rd = Ra + Shift(Rb)
SUB	R	00	001	Rd	Ra	Shift	Rb	Z, N, C, V	Rd = Ra - Shift(Rb)
ADDC	R	00	010	Rd	Ra	Shift	Rb	Z, N, C, V	Add with Carry, Rd = Ra + Shift(Rb) + C
SUBC	R	00	011	Rd	Ra	Shift	Rb	Z, N, C, V	Sub with Carry, Rd = Ra - Shift(Rb) - C
AND	R	00	100	Rd	Ra	Shift	Rb	Z, N	Rd = Ra & Shift(Rb)
ANDBB	R	00	101	Rd	Ra	Shift	Rb	Z, N	Rd = Ra & Bar(Shift(Rb))
OR	R	00	110	Rd	Ra	Shift	Rb	Z, N	Rd = Ra | Shift(Rb)
ORBB	R	00	111	Rd	Ra	Shift	Rb	Z, N	Rd = Ra | Bar(Shift(Rb)
IMMEDIATE		Op[15:14]	ALUop[13:11]	Rd[10:8]	Ra[7:5]	IMM5[4:0]		Flags	(Merges Shift & Src2 into Imm5)
ADDI	I	01	000	Rd	Ra	sximm5		Z, N, C, V	Rd = Ra + SignExt(Imm5)
SUBI	I	01	001	Rd	Ra	sximm5		Z, N, C, V	Rd = Ra - SignExt(Imm5)
IMM REG LOAD		OpCode[15:11]		DST[10:8]		IMM8[7:0]			
LDI		10000		Rd	sximm8			-	Rd = SignExt(Imm8)
LUI		10001		Rd	sximm8			-	Rd = TailExt(Imm8)

Recap on CPU: A Factory of Data

- What we have done: The processor (controller and datapath)
- What we are going to accomplish: Memory, IO, and advanced controller
- For now, we support sequential arithmetic instructions and load register instructions.
- What we need to do:
  - To support instruction and data memory.
  - Use PC for automated execution.
  - Support conditional execution: jumping and branching



Recap: Coursework Goal

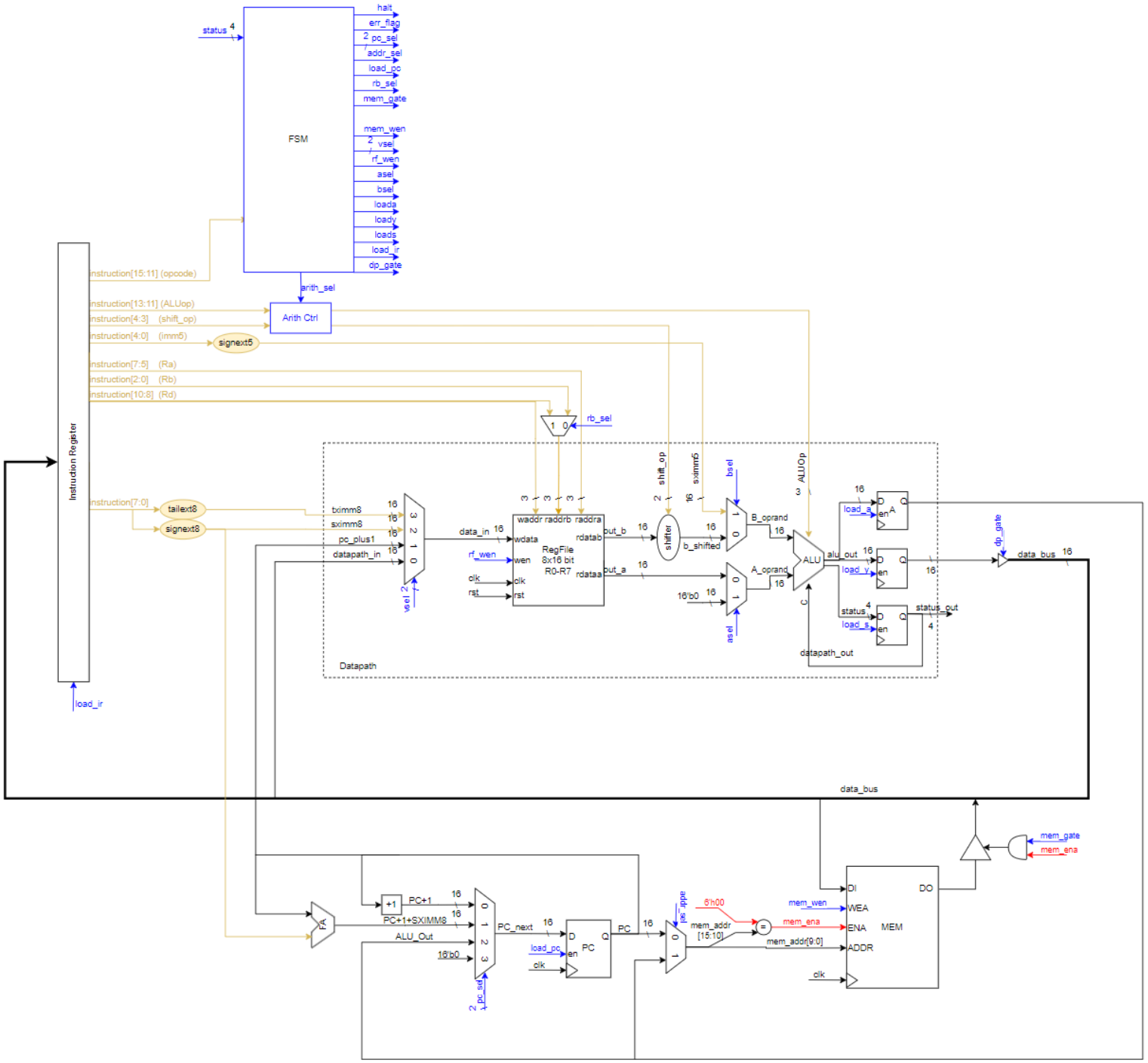
- 1. Complete the CPU design by filling in the template project
- 2. Prepare your own program, execute it and show the demo in VIVA
- 3. Register for VIVA timeslot: HAD26LAB.xlsx
- 4. Write your report

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U
1											Week 6										
2		Slot1	Slot 2	Slot3	Slot4	Slot5	Slot6	Slot7	Slot8	Slot9	Slot10	Slot11	Slot12	Slot13	Slot14	Slot15	Slot16	Slot17	Slot18	Slot19	Slot20
3	Zihao																				
4	Ali																				
5	Stefan																				
6																					
7																					
8																					
9																					
10											Week 7										
11		Slot1	Slot 2	Slot3	Slot4	Slot5	Slot6	Slot7	Slot8	Slot9	Slot10	Slot11	Slot12	Slot13	Slot14	Slot15	Slot16	Slot17	Slot18	Slot19	Slot20
12	Zihao																				
13	Ali																				
14	Stefan																				
15																					

Coursework Checklist

- ❑ Finish CPU design
- ❑ Verify with Verilog Autograder With Waveforms
- ❑ Write your own assembly program
  - Step 1: refer to case 1 and case 2, write your own cases
  - Step 2: use assembler to compile assembly program to binaries
  - Step 3: prepare your own mem_ref.txt and reg_ref.hex, based on your program. These files show the final results in memory and registers.
- ❑ Make a testbench with your program
  - Step 4: in cpu_tb.sv, add your own program in the test cases. Refer to existing cases
- ❑ Download to BASYS3 for Demo
  - Step 5: in Vivado, add your compiled .coe file to memory IP, generate bitstream and download to BASYS3
- ❑ Write your report
- ❑ Wrap them up into a zip file, them submit to KEATS

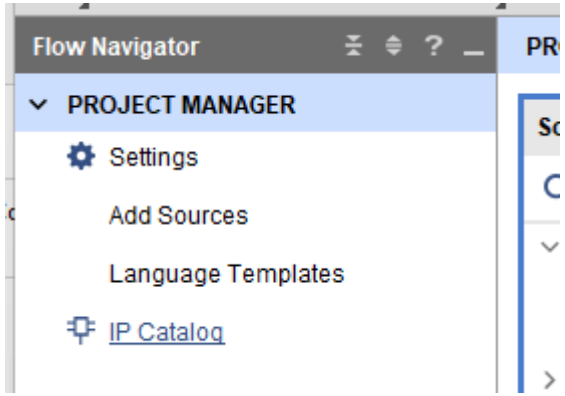
Complete Architecture: Overview

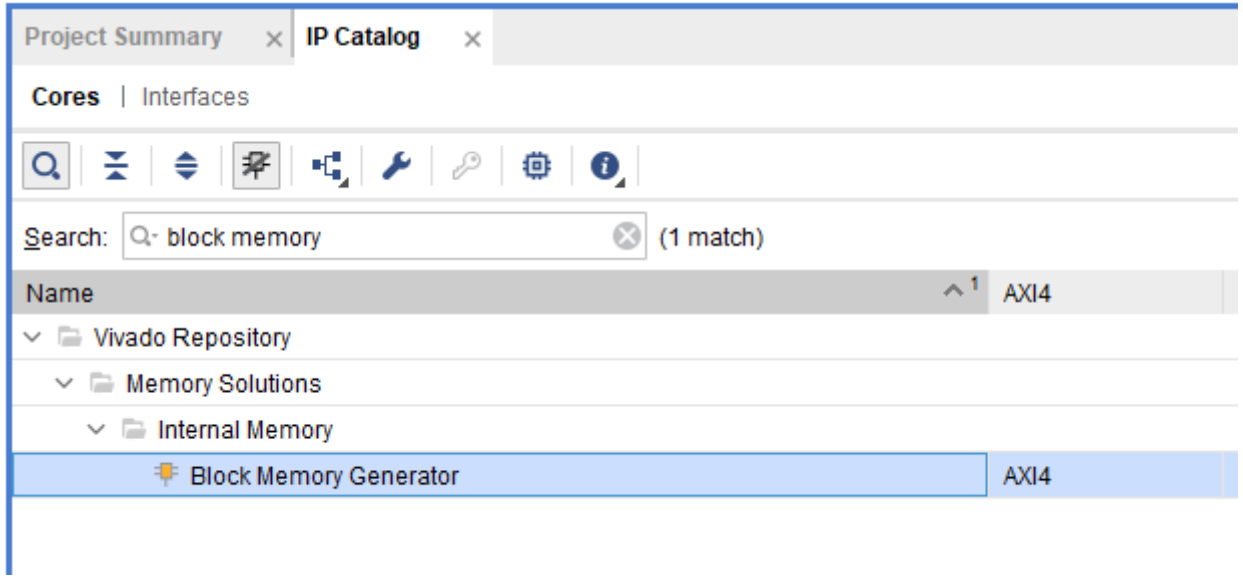


Memory

WHERE PROGRAM AND DATA STORE

Memory Module: Using Vivado Block Memory





Memory Module: Using Vivado Block Memory

Customize IP

Block Memory Generator (8.4)

Documentation | IP Location | Switch to Defaults

IP Symbol | **Power Estimation**

Show disabled ports

BRAM_PORTA
- addra[3:0]
- clka
- dina[17:0]
- douta[17:0]
- ena
- wea[0:0]

Component Name: blk_mem_gen_1

Basic | **Port A Options** | **Other Options** | **Summary**

Interface Type: Native — Generate address interface with 32 bits

Memory Type: Single Port RAM — Common Clock

ECC Options

ECC Type: No ECC

Error Injection Pins: Single Bit Error Injection

Write Enable

Byte Write Enable

Byte Size (bits): 9

Algorithm Options

Defines the algorithm used to concatenate the block RAM primitives.
Refer datasheet for more information.

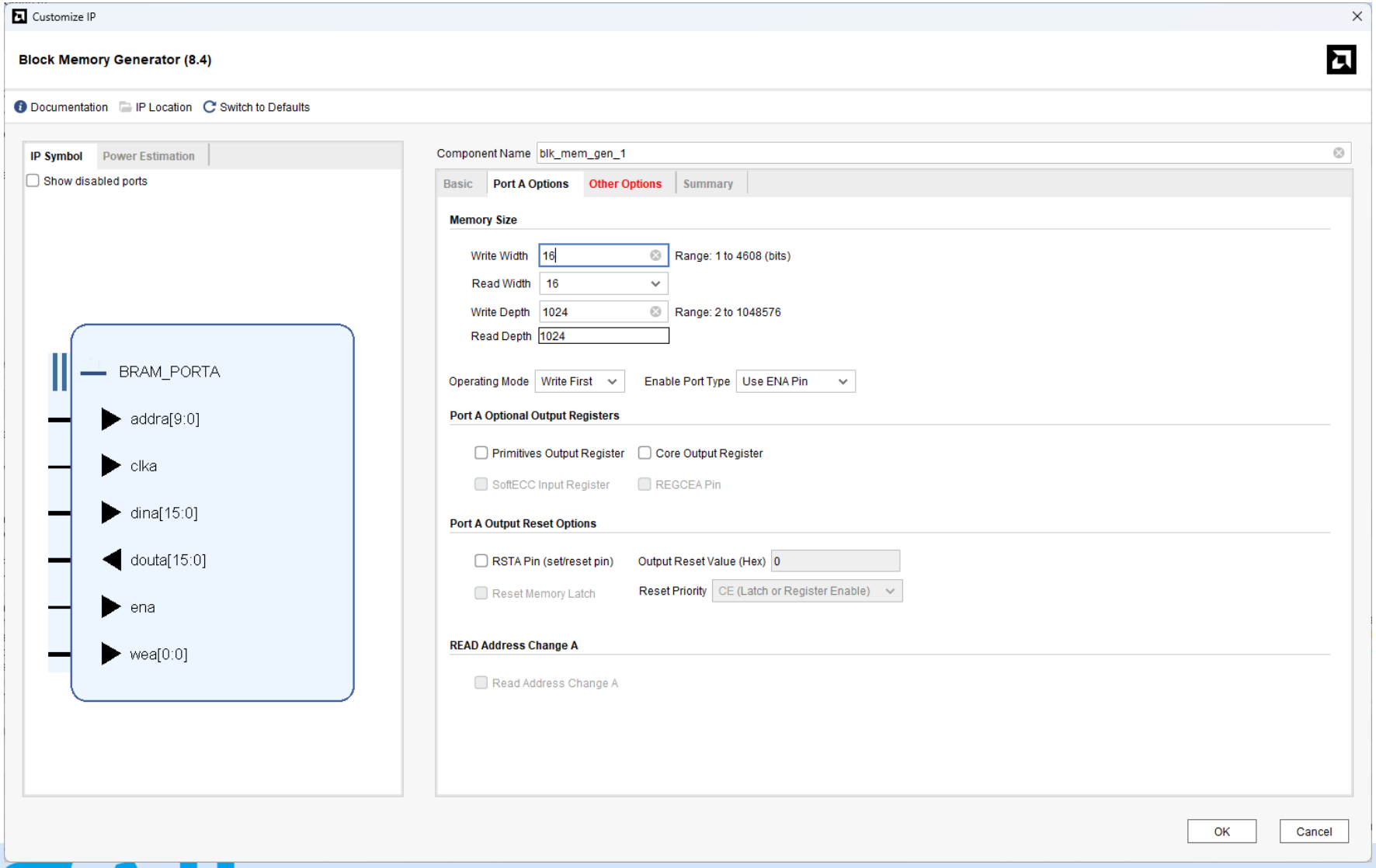
Algorithm: Minimum Area

Primitive: 8kx2

OK | Cancel

- RISKing16
  - .git
  - ip
  - prj
  - sim
  - src

Memory Module: Using Vivado Block Memory



Memory Module: Using Vivado Block Memory

- **Write First Mode**: In WRITE_FIRST mode, the input data is simultaneously written into memory and driven on the data output, as shown in Figure 3-9. This transparent mode offers the flexibility of using the data output bus during a Write operation on the same port.

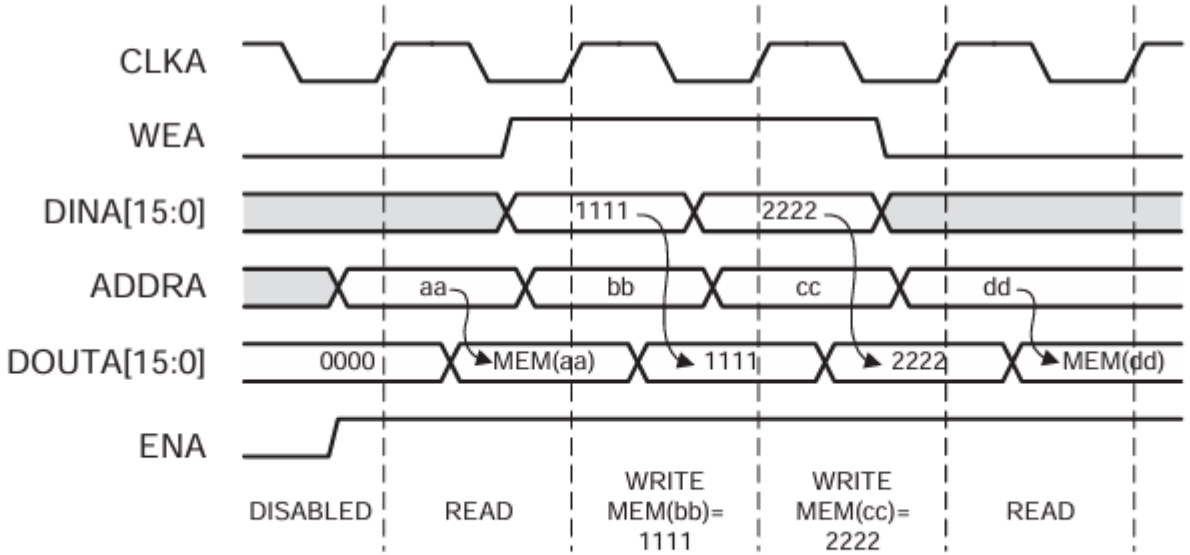
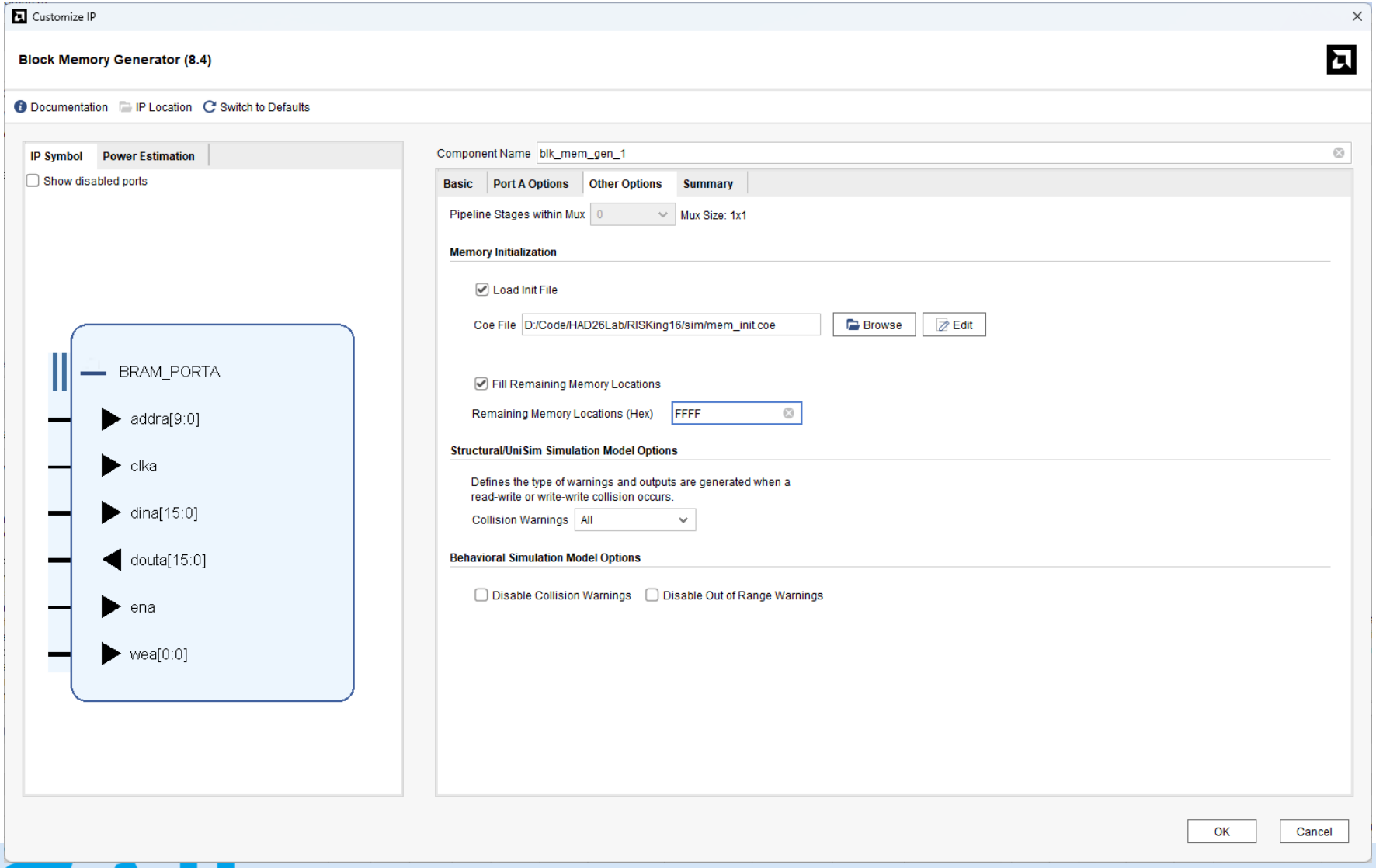


Figure 3-9: **Write First Mode Example**

https://docs.amd.com/v/u/en-US/pg058-blk-mem-gen

Memory Module: Using Vivado Block Memory

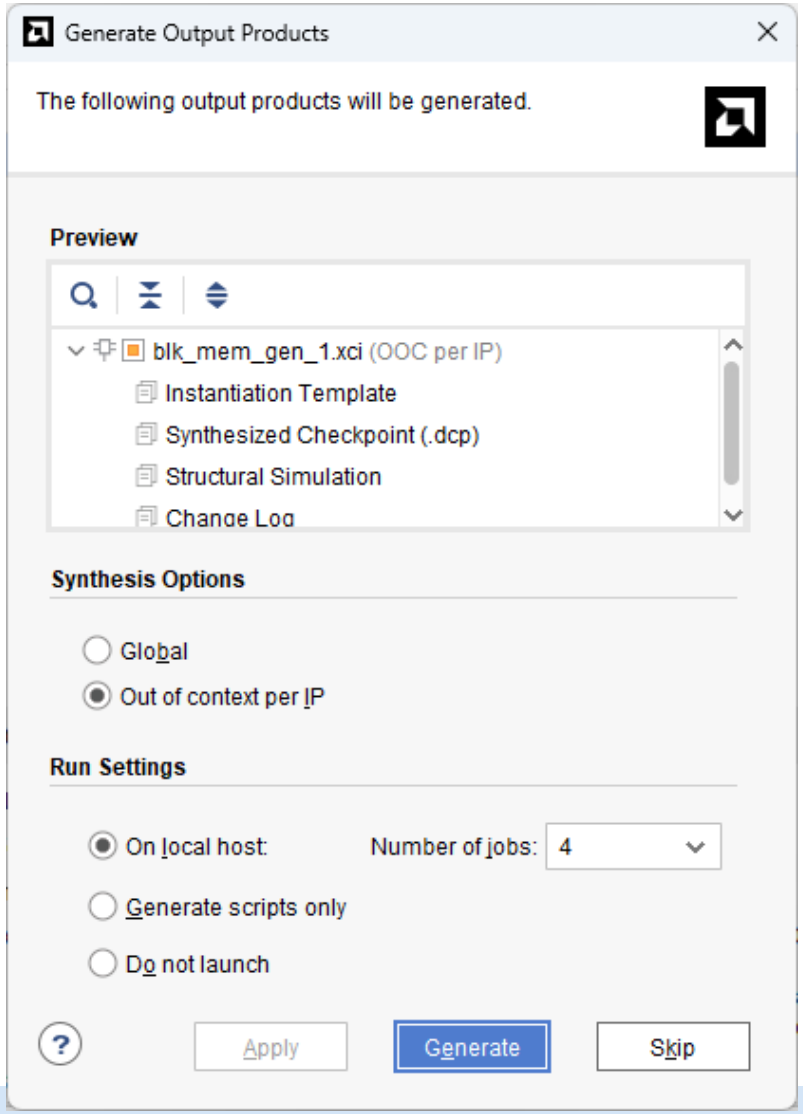


Memory Module: Using Vivado Block Memory

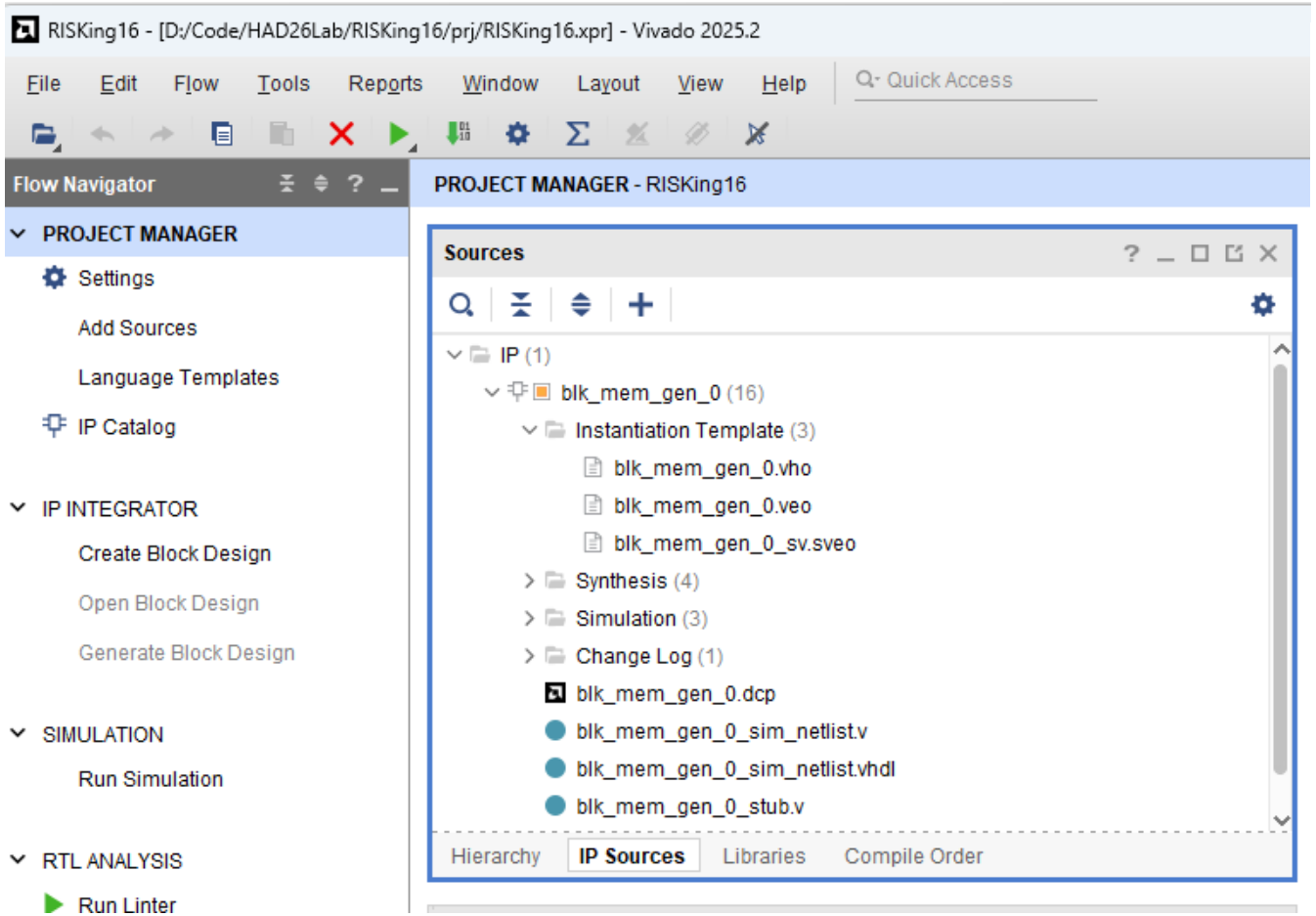
sim > mem_init.coe

```
memory_initialization_radix=16;
memory_initialization_vector=
810A,
820B,
C007,
FFFF,
FFFF,
FFFF,
FFFF,
FFFF,
FFFF,
FFFF,
FFFF,
0022,
FFFF,
FFFF,
FFFF,
```

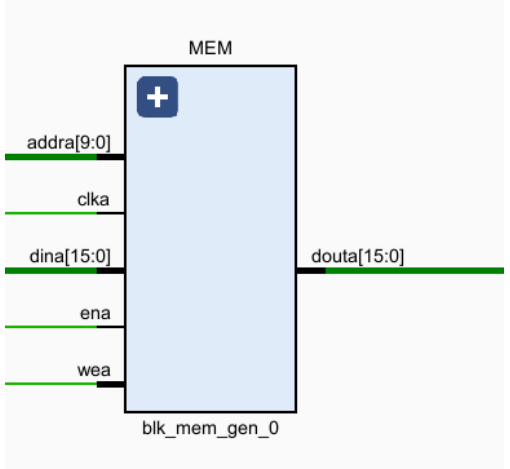
Memory Module: Using Vivado Block Memory



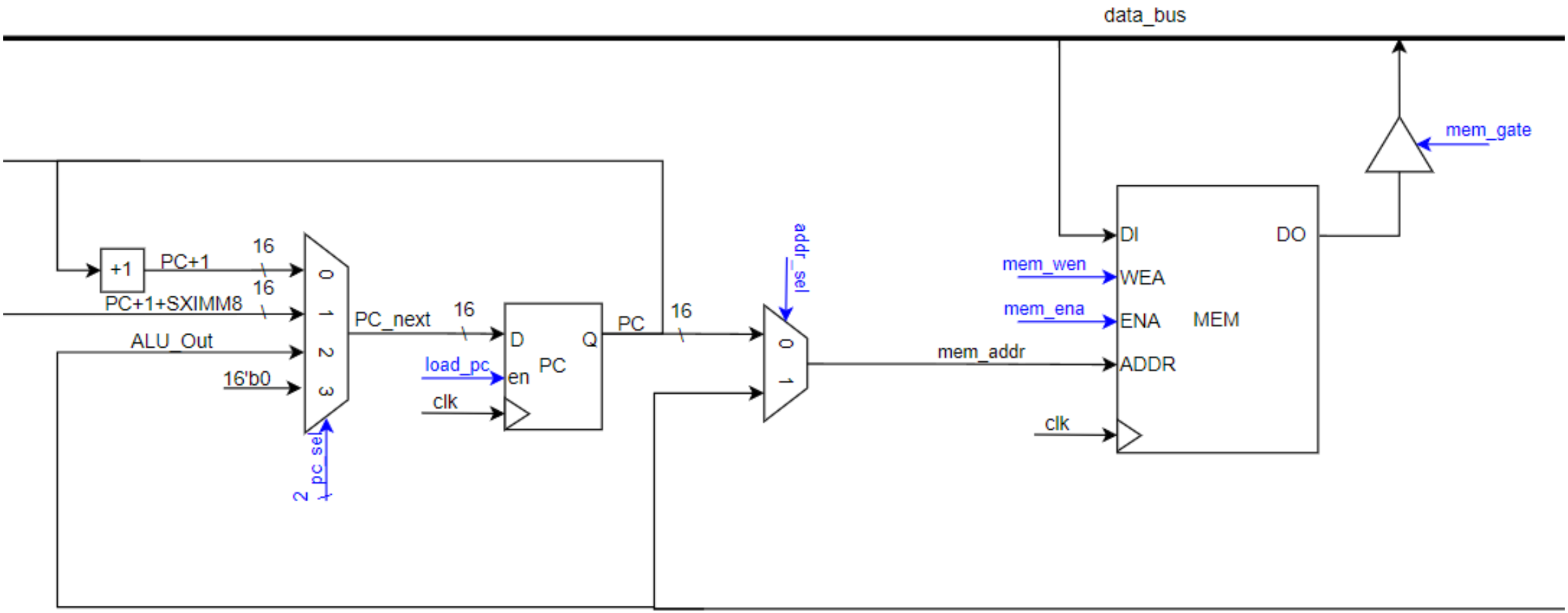
Memory Module: Using Vivado Block Memory




```
55
56 //----------- Begin Cut here for INSTANTIATION Template ---// INST_TAG
57 blk_mem_gen_0 your_instance_name (
58   .clka(clka),     // input wire clka
59   .ena(ena),       // input wire ena
60   .wea(wea),       // input wire [0 : 0] wea
61   .addra(addra),   // input wire [9 : 0] addra
62   .dina(dina),     // input wire [15 : 0] dina
63   .douta(douta)   // output wire [15 : 0] douta
64 );
```



Program Counter and Memory

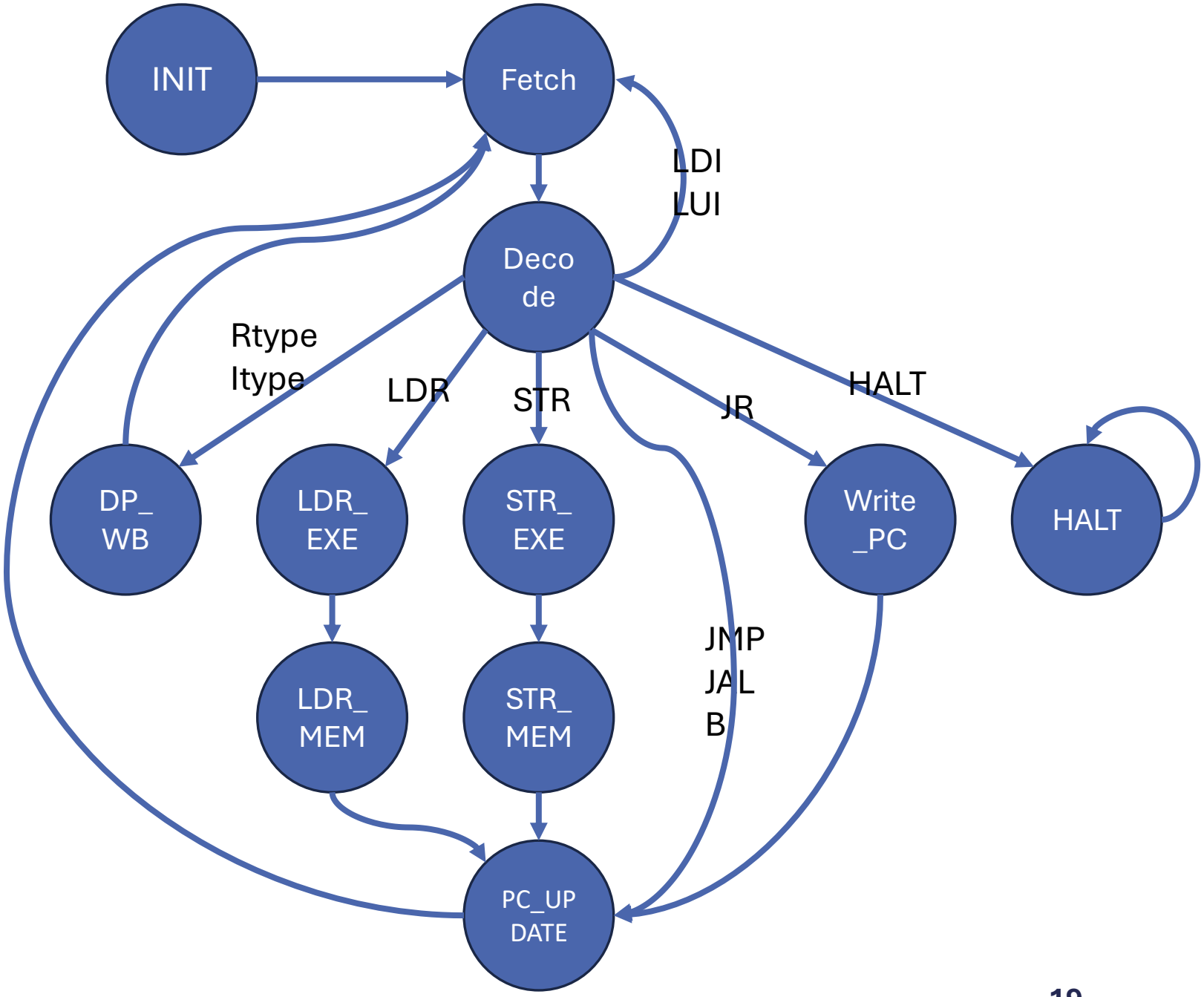


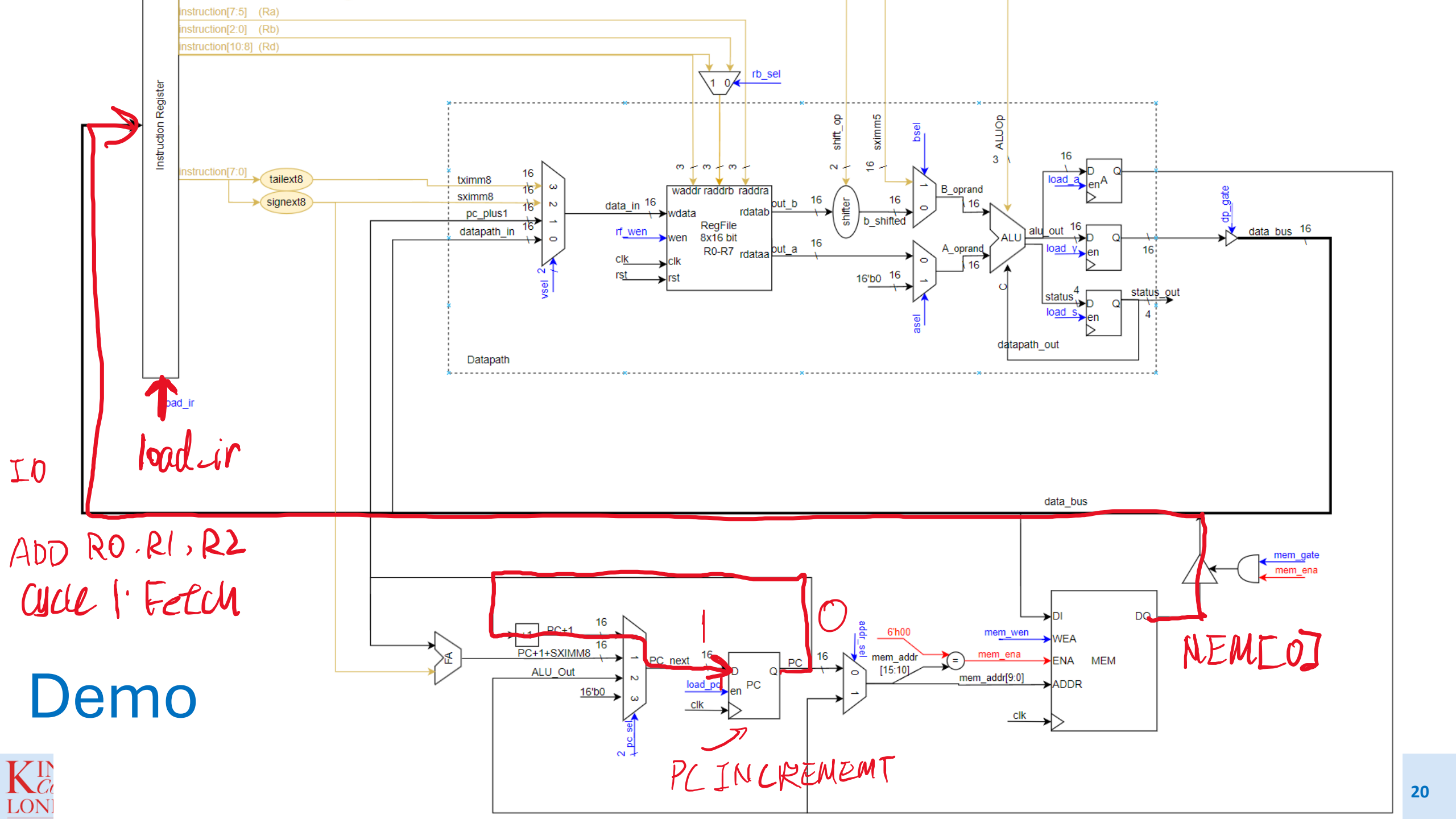
FSM

TO SUPPORT MEMORY ACCESS AND BRANCHING

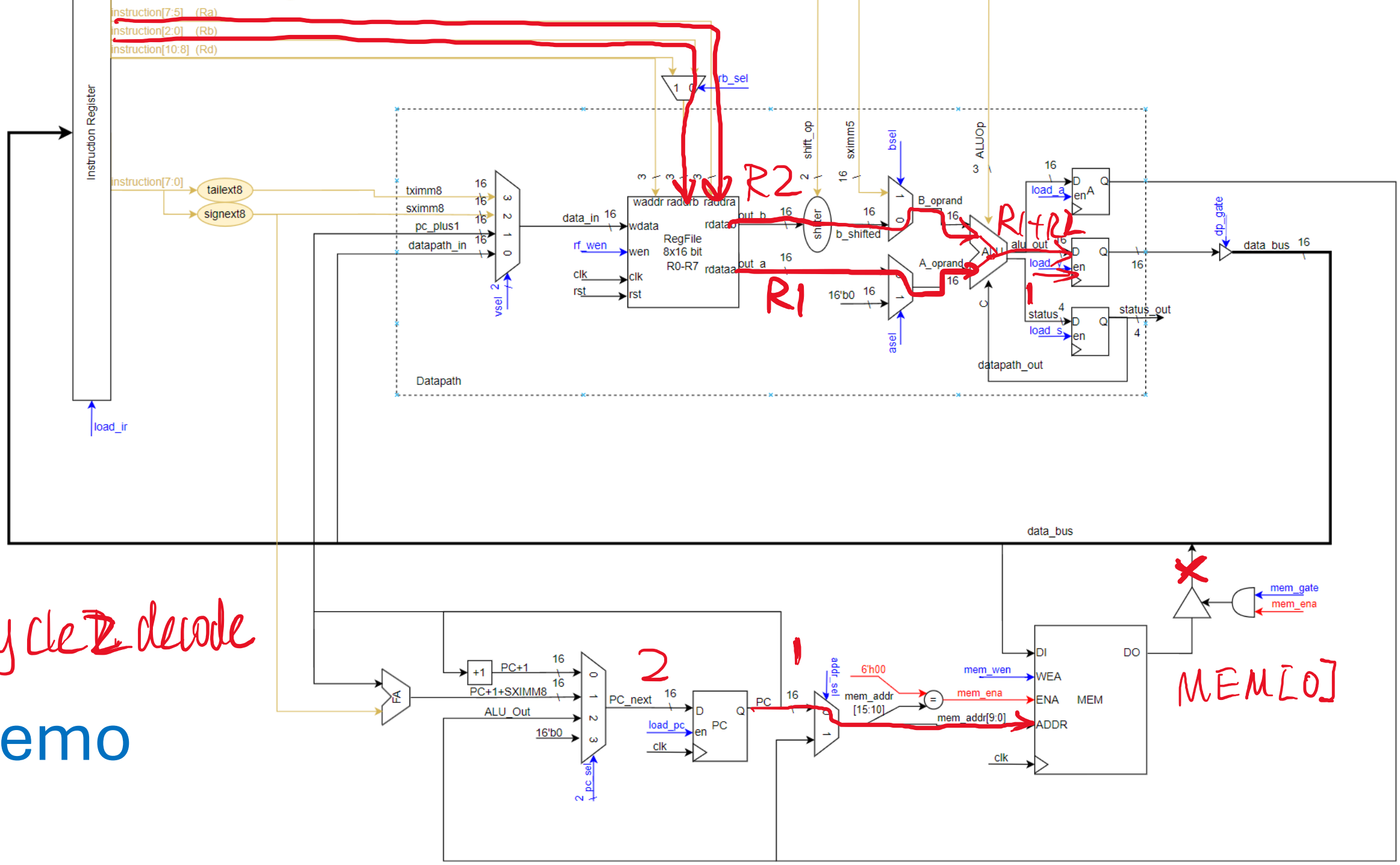
Example FSM

You can implement FSM in your own way. Especially if you choose to use a different datapath design.

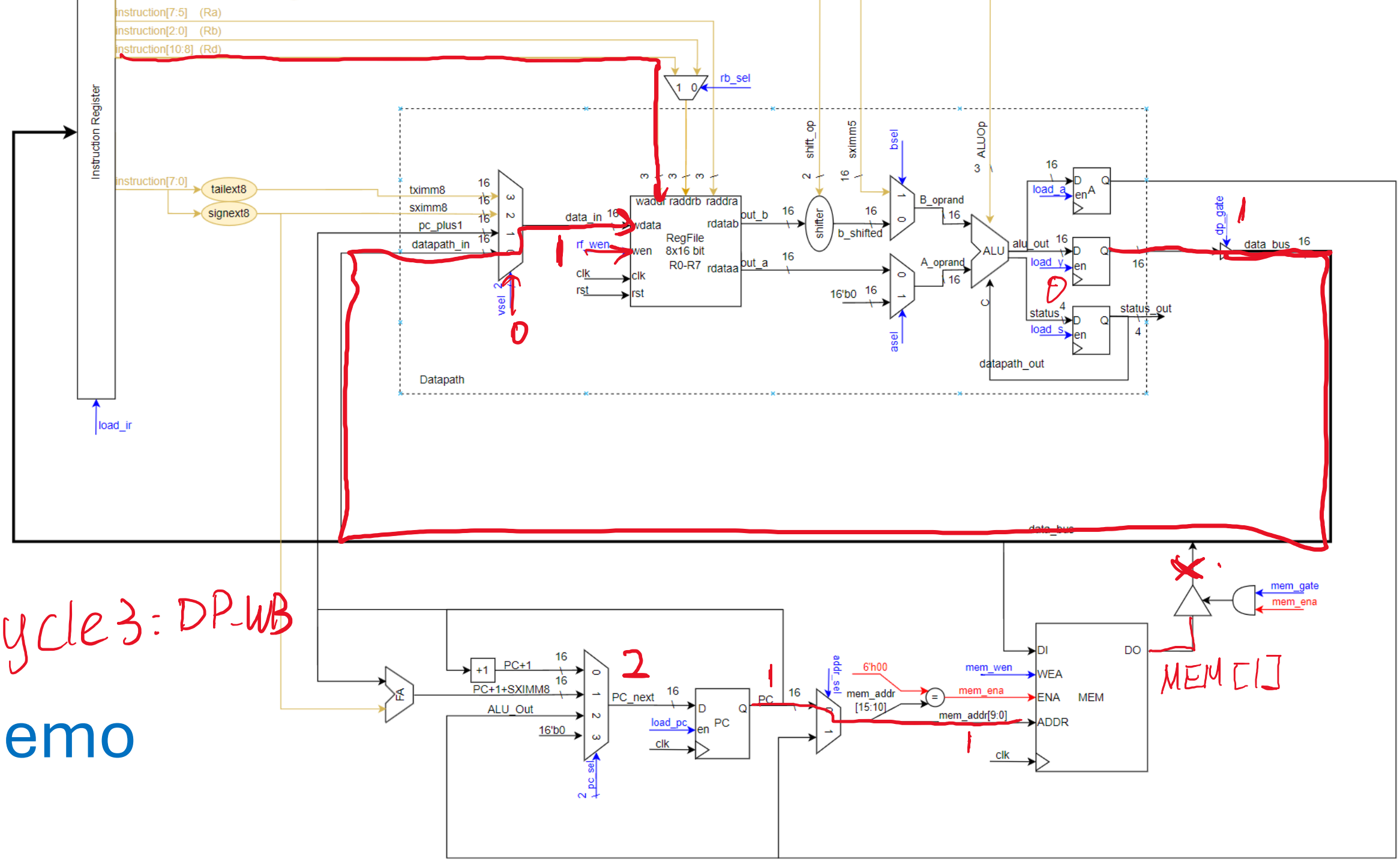



Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
instruction[7:0]
tailext8
signext8
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
waddr raddr raddra
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
shifter
sximm5
b_shifted
bsel
B_oprand
A_oprand
16'b0
asel
ALUOp
ALU
alu_out
load_a
load_y
load_s
status
status_out
datapath_out
Datapath
dp_gate
data_bus
load_ir
IO
ADD R0, R1, R2
Cycle 1: Fetch
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
clk
PC INCREMENT
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
mem_ena
DI
WEA
ENA
ADDR
MEM
DO
mem_gate
NEM[0]

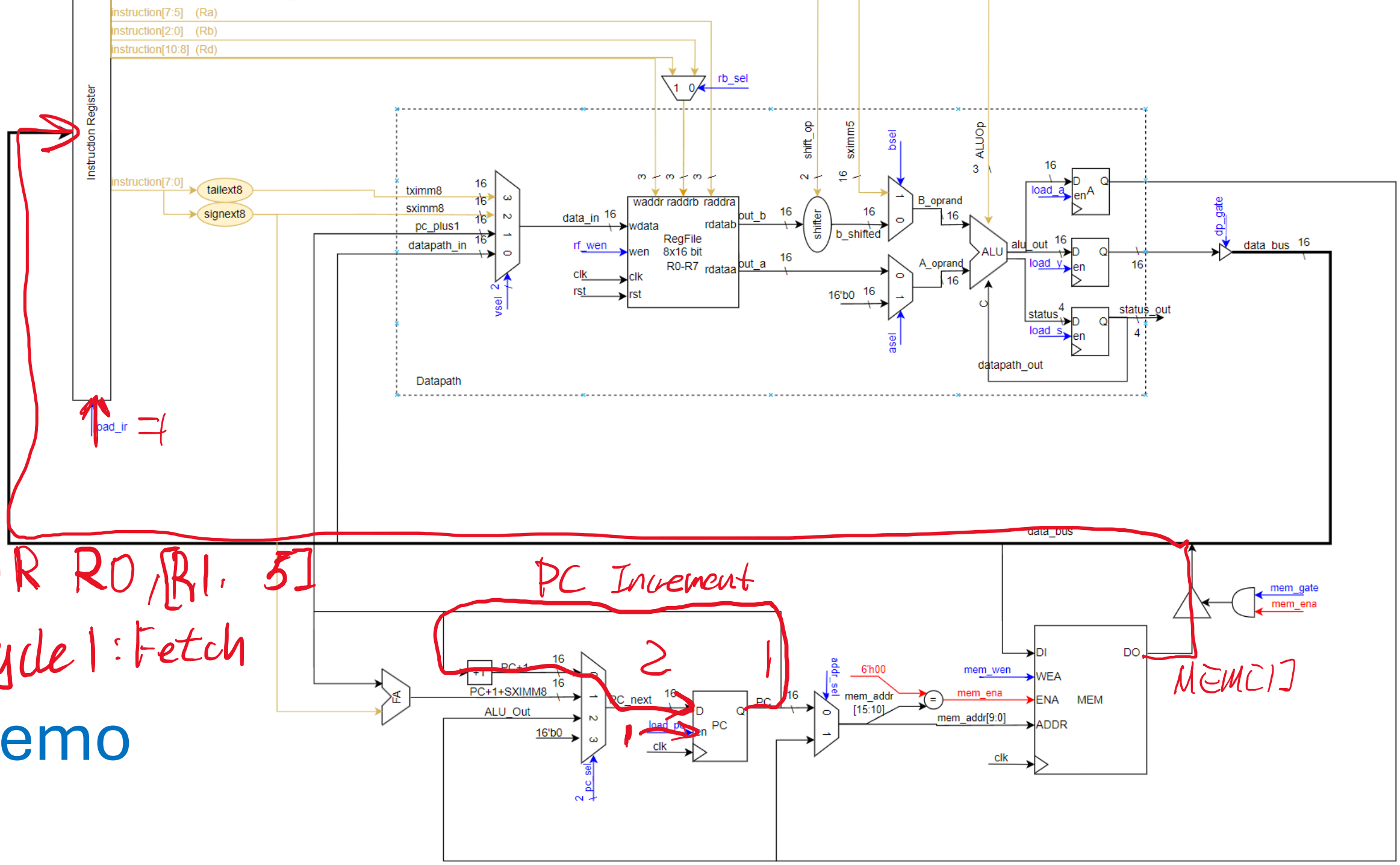
Demo


instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
Instruction Register
instruction[7:0]
tailext8
signext8
load_ir
Datapath
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
wdata
rf_wen
wen
clk
rst
waddr
raddr
RegFile
8x16 bit
R0-R7
rdata
out_b
out_a
shift_op
sximm5
shifter
b_shifted
bsel
asel
16'b0
B_oprand
A_oprand
ALUOp
ALU
alu_out
load_a
load_b
load_s
status
status_out
datapath_out
dp_gate
data_bus
R2
R1
R1+R2
1
2
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
FA
PC_next
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
mem_ena
DI
WEA
ENA
ADDR
DO
MEM
mem_gate
mem_ena
MEM[0]
cycle 2 decode

Demo


Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
instruction[7:0]
tailext8
signext8
load_ir
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
wadr raddr raddra
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
sximm5
shifter
b_shifted
bsel
asel
16'b0
B_oprand
A_oprand
ALUOp
ALU
alu_out
load_a
load_y
status
load_s
status_out
datapath_out
Datapath
dp_gate
data_bus
mem_gate
mem_ena
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
DI
WEA
ENA
ADDR
DO
MEM
clk
MEM[1]
cycle3: DP-WB

Demo



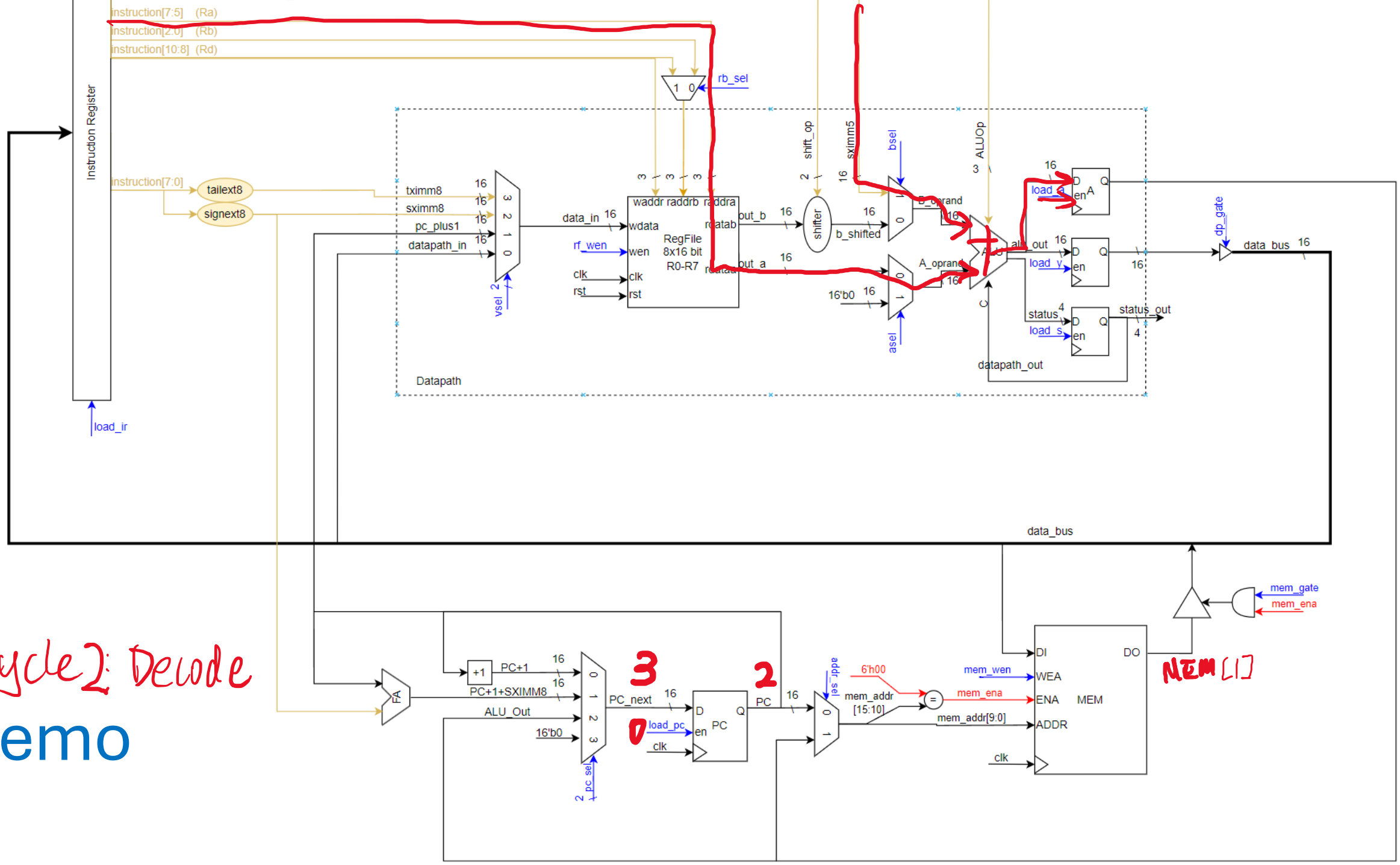
LDR R0,[R1, 5]

Cycle 1: Fetch

PC Increment

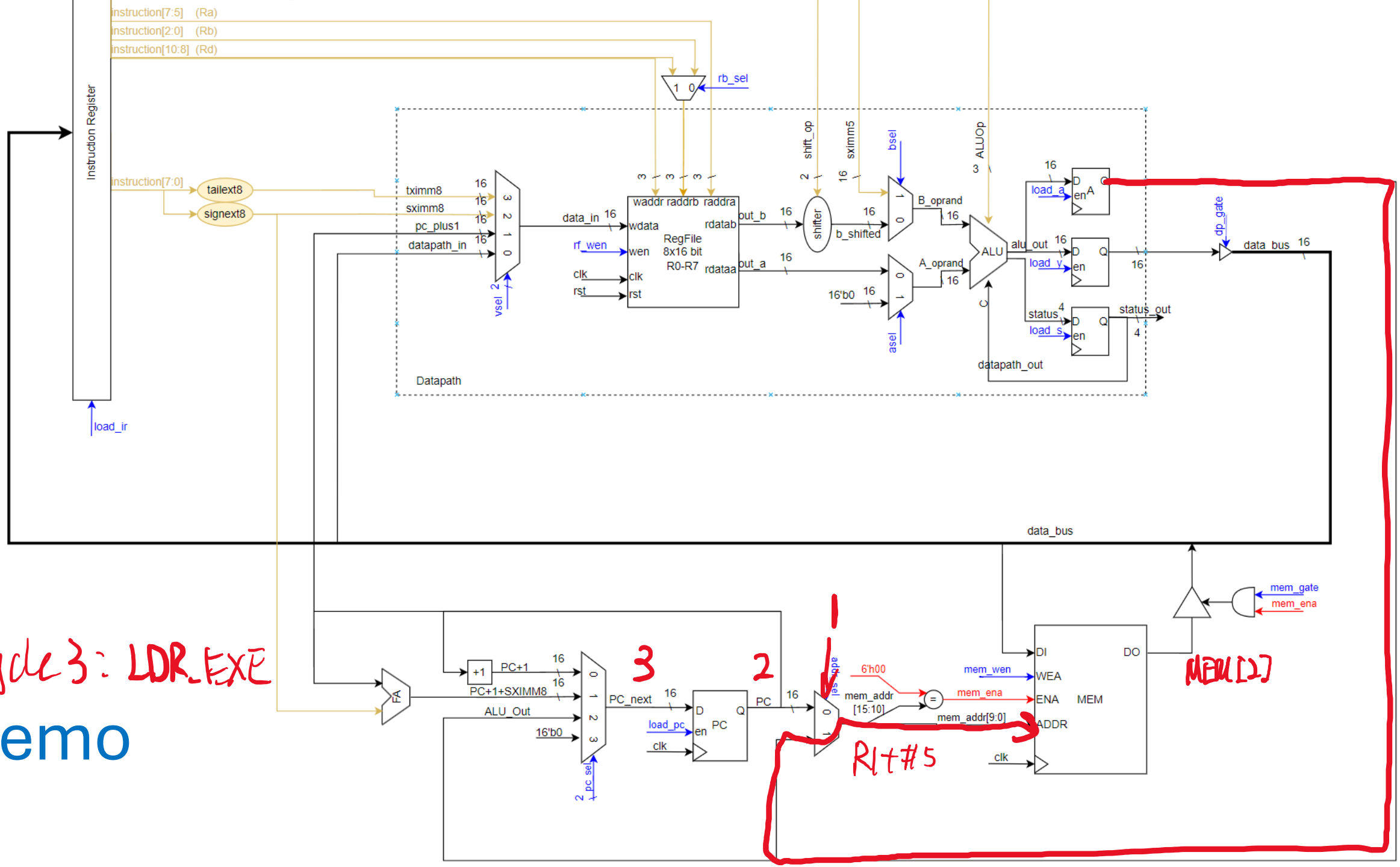
M[M[1]]

Demo



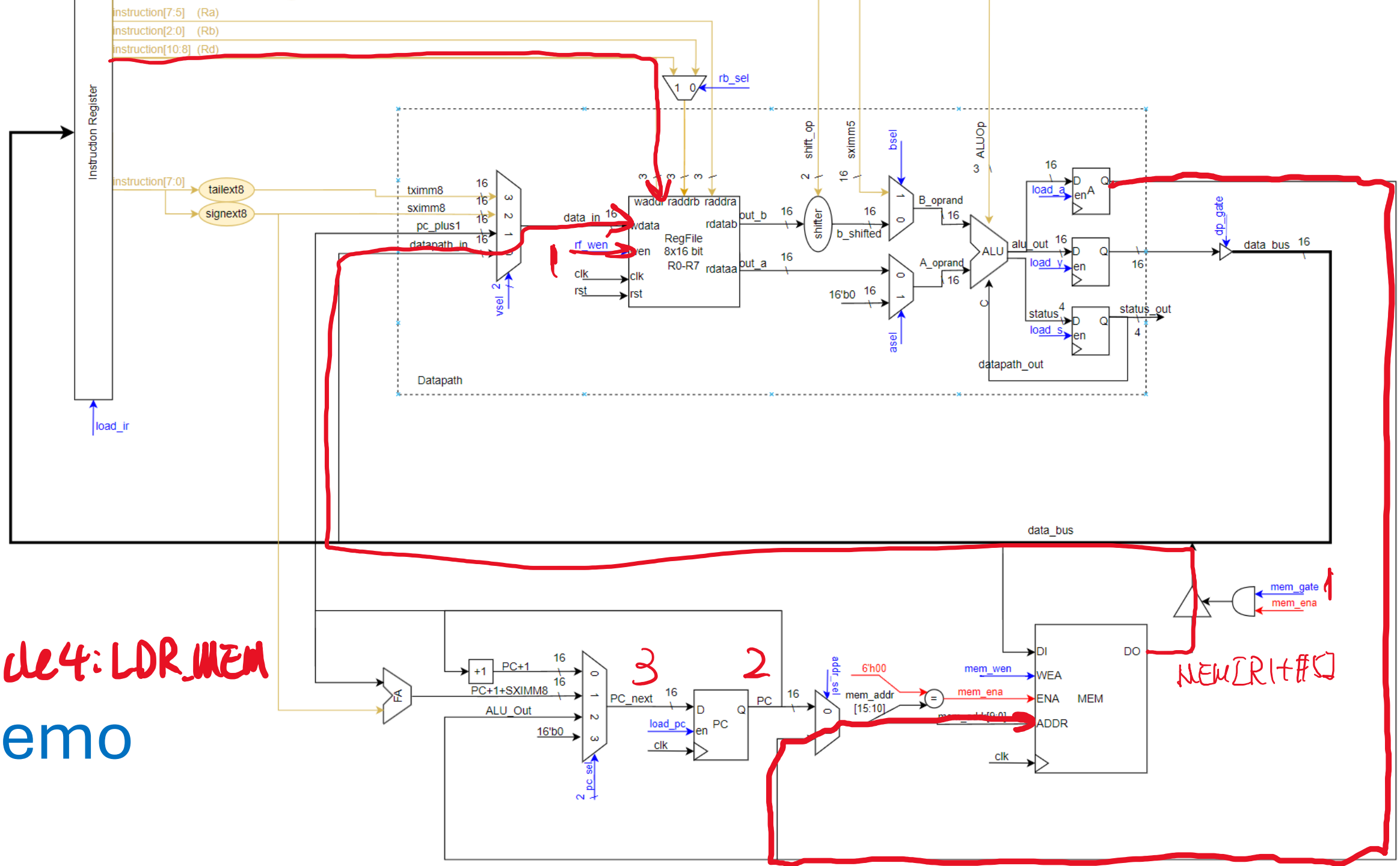
Cycle2: Decode

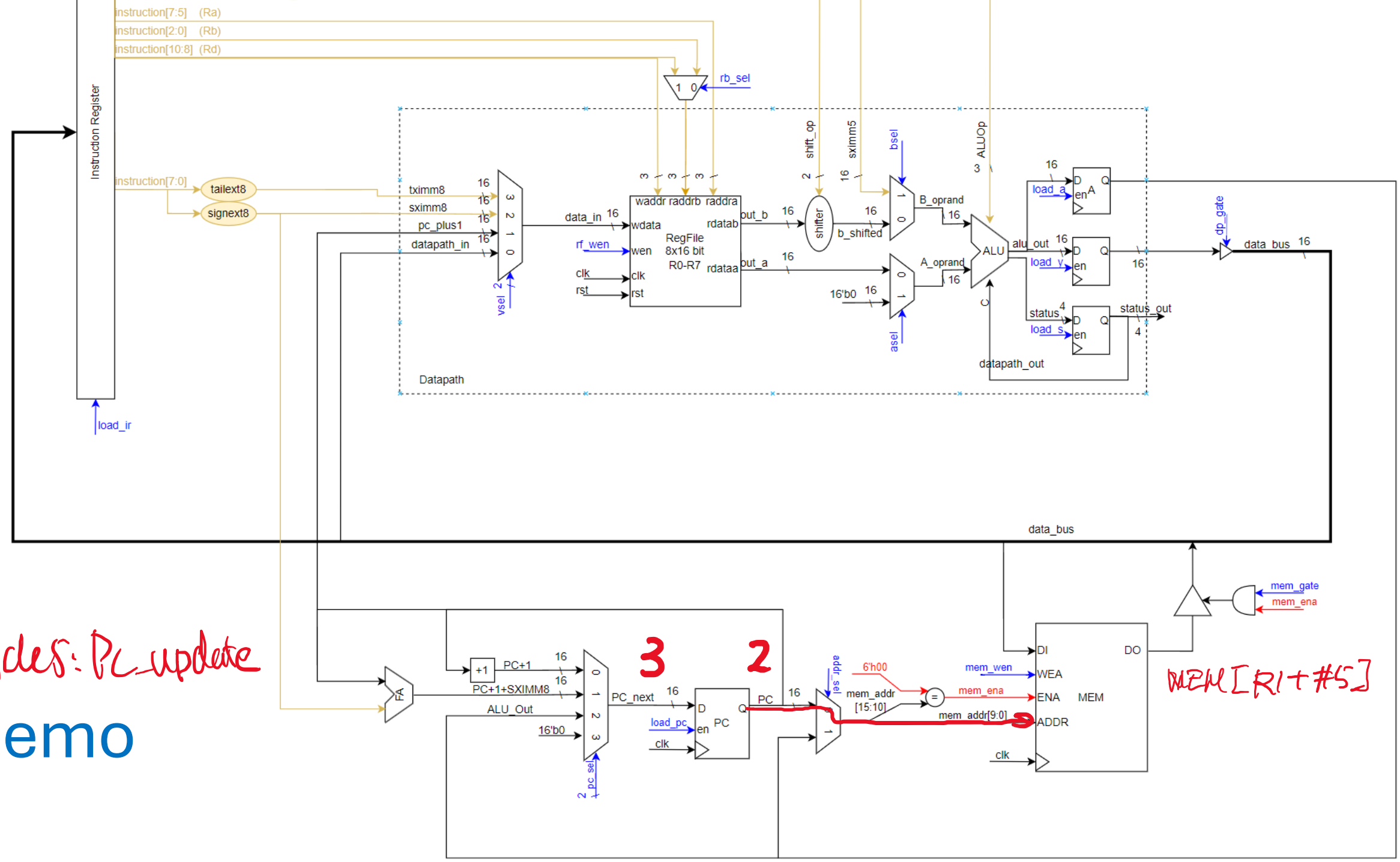
Demo



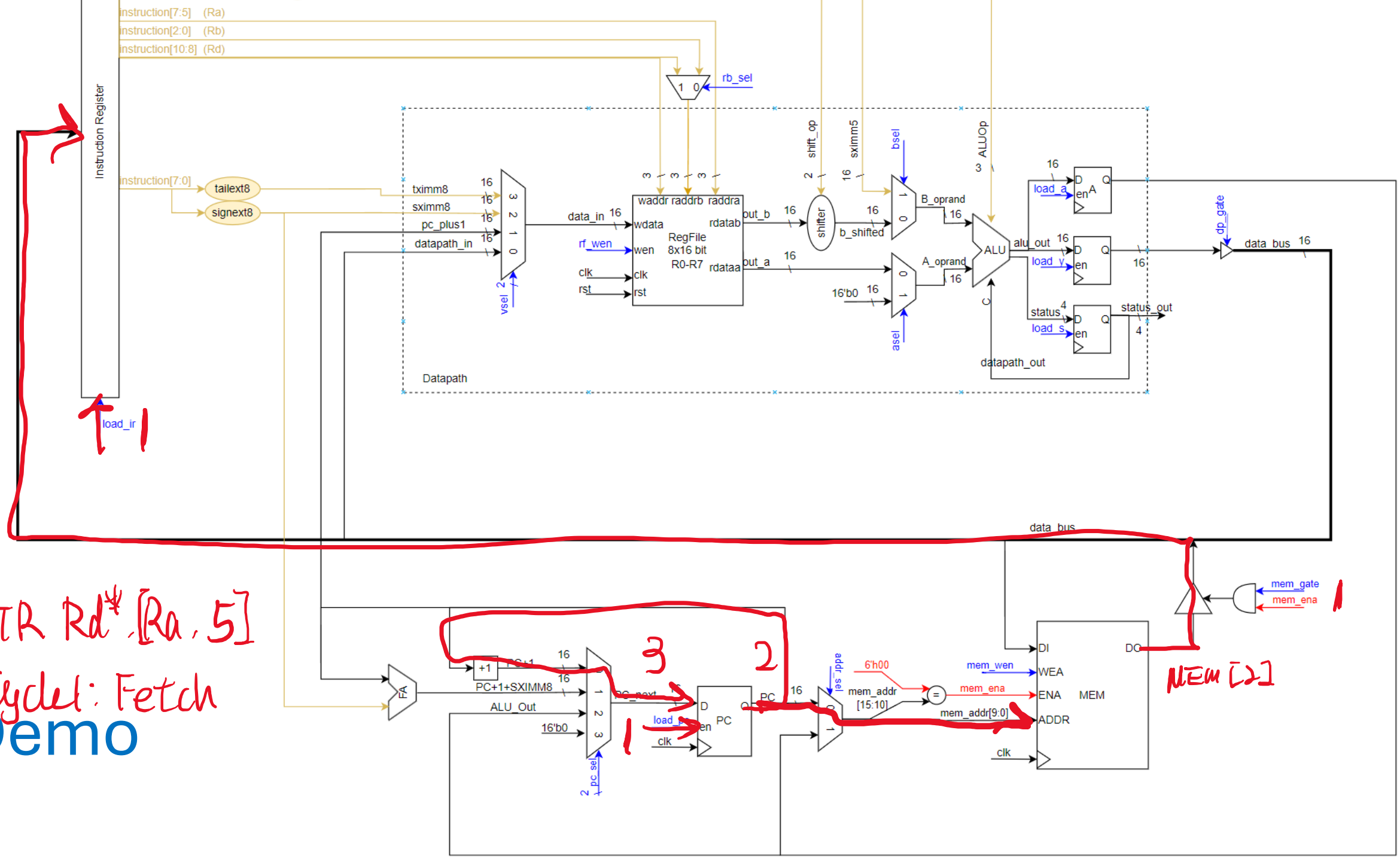
Cycle 3: LDR_EXE

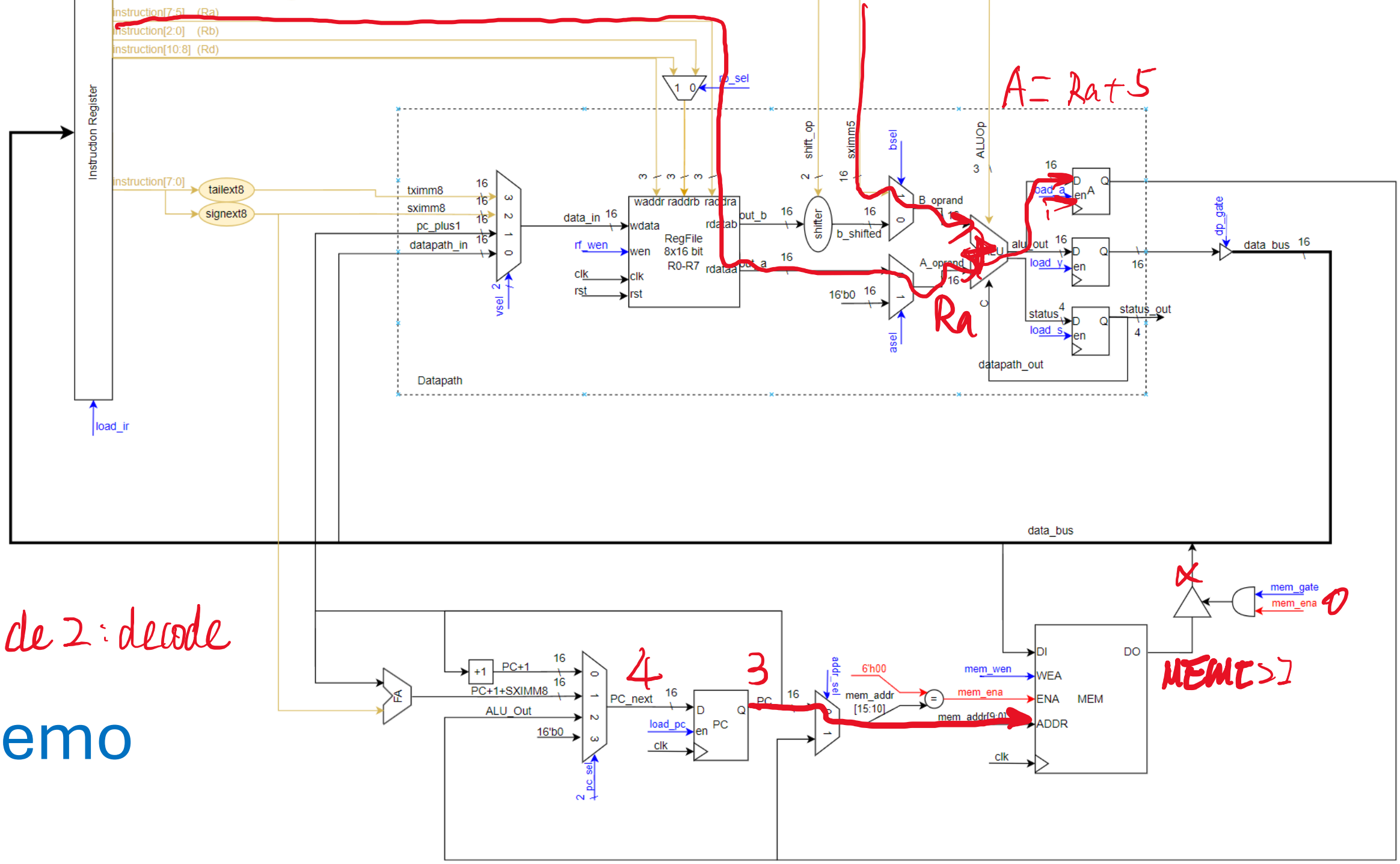
Demo


Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
instruction[7:0]
tailext8
signext8
rb_sel
shift_op
sximm5
bsel
ALUOp
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
wadr raddrb raddra
wdata
rf_wen
wen
clk
rst
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shifter
b_shifted
16'b0
asel
B_oprand
A_oprand
ALU
alu_out
load_a
load_y
load_s
status
status_out
dp_gate
data_bus
datapath_out
Datapath
load_ir
mem_gate
mem_ena
+1
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_wen
DI
WEA
ENA
ADDR
DO
MEM
Cycle4: LDR_MEM
MEM[R1+#5]
Demo


Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
instruction[7:0]
tailext8
signext8
load_ir
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
waddr raddr b raddr a
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
shifter
sximm5
b_shifted
bsel
B_oprand
16'b0
asel
A_oprand
ALUOp
ALU
alu_out
load_a
load_y
status
load_s
status_out
datapath_out
Datapath
dp_gate
data_bus
mem_gate
mem_ena
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
DI
WEA
ENA
ADDR
DO
MEM
Cycles: PC_update
3
2
MEM[R1+#5]

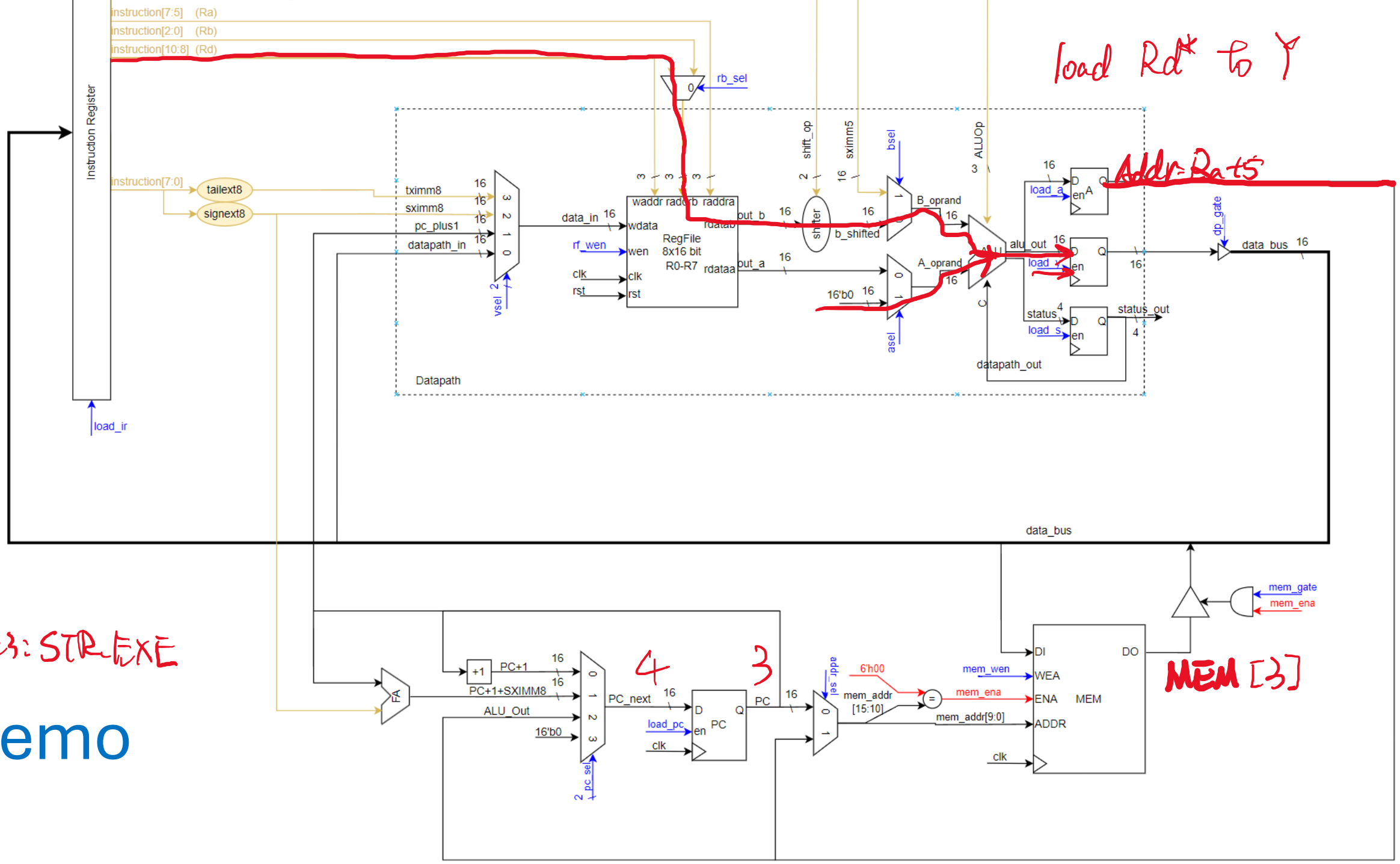
Demo


instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
Instruction Register
instruction[7:0]
tailext8
signext8
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
waddr raddr raddra
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
shifter
sximm5
b_shifted
bsel
B_oprand
16'b0
asel
A_oprand
ALUOp
ALU
alu_out
load_a
load_y
status
load_s
status_out
dp_gate
data_bus
datapath_out
Datapath
load_ir
mem_gate
mem_ena
DI
DO
WEA
ENA
ADDR
MEM
mem_wen
mem_addr[9:0]
mem_addr
[15:10]
6'h00
addr_sel
PC
load_pc
clk
+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
FA
STR Rd*,[Ra,5]
Cycle1: Fetch
Demo
MEM[2]

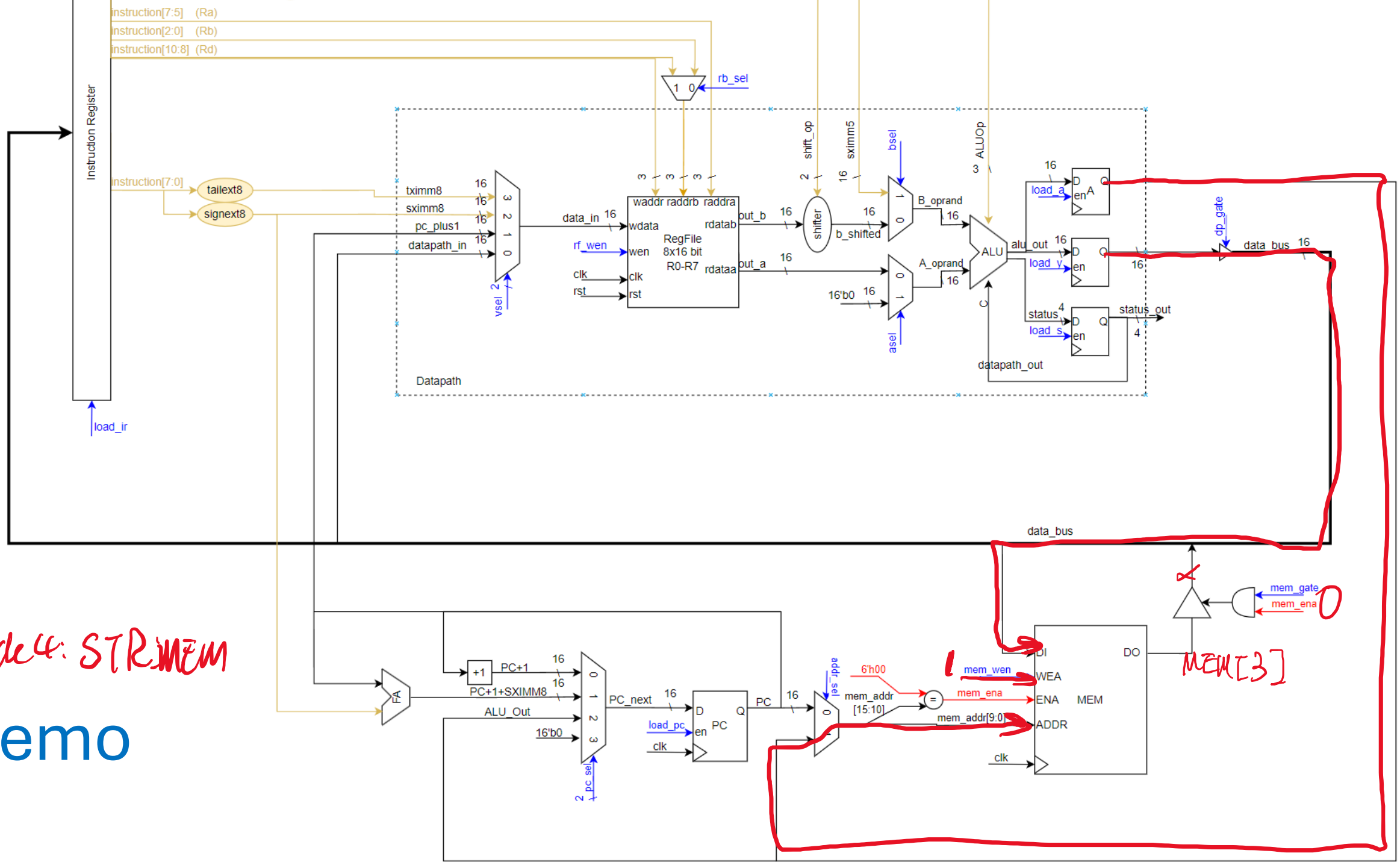


cycle 2: decode

Demo

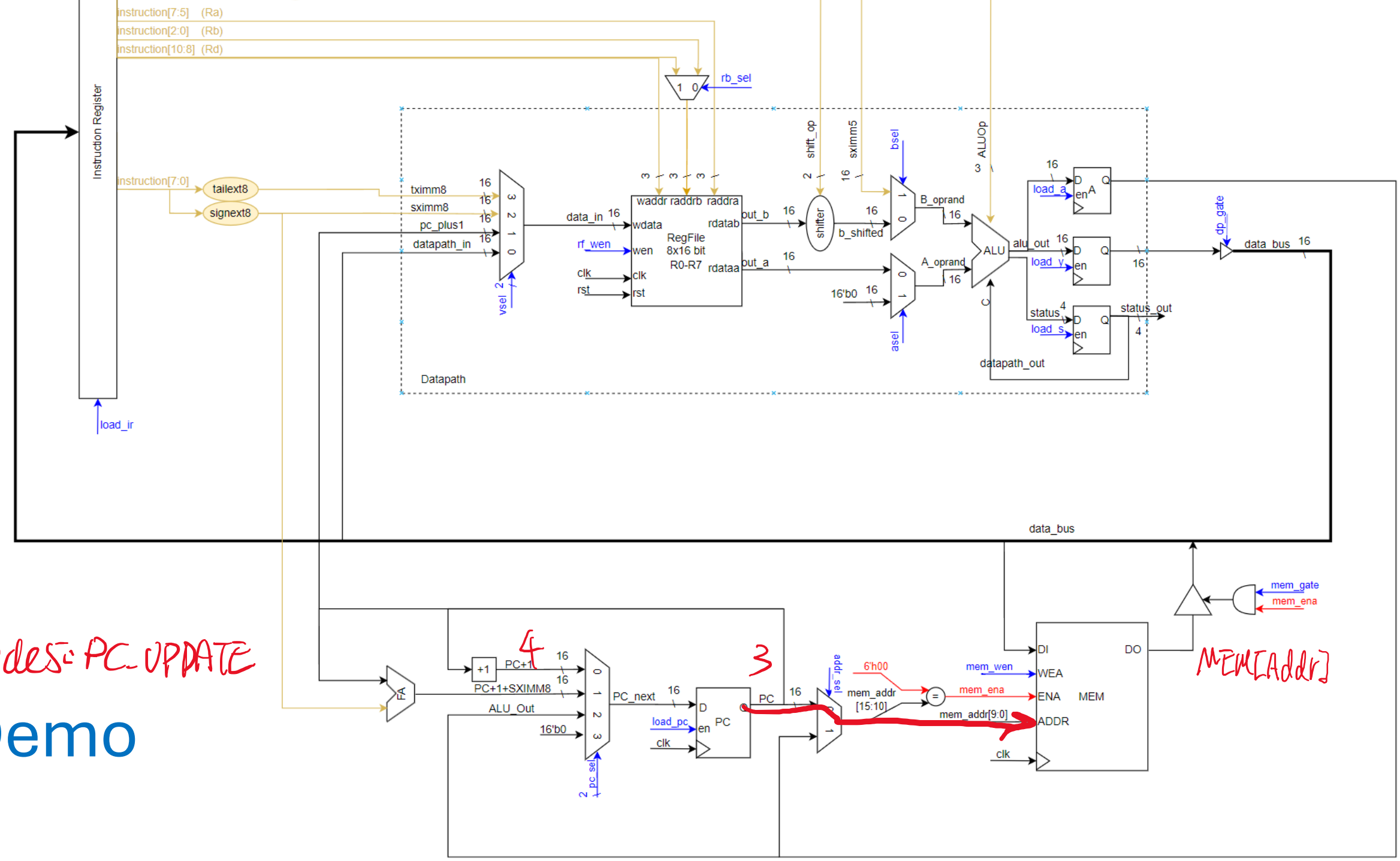

load Rd* to Y
Addr=Ra+5
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
Instruction Register
rb_sel
shift_op
sximm5
bsel
ALUOp
instruction[7:0]
tailext8
signext8
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
wdata
rf_wen
wen
clk
rst
waddr
raddra
rdataa
RegFile
8x16 bit
R0-R7
out_b
out_a
shifter
b_shifted
B_oprand
A_oprand
16'b0
asel
load_a
load_b
alu_out
status
load_s
status_out
datapath_out
dp_gate
data_bus
Datapath
load_ir
data_bus
mem_gate
mem_ena
Cycle3: STR-EXE
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
4
3
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
DI
WEA
ENA
ADDR
DO
MEM
MEM[3]

Demo

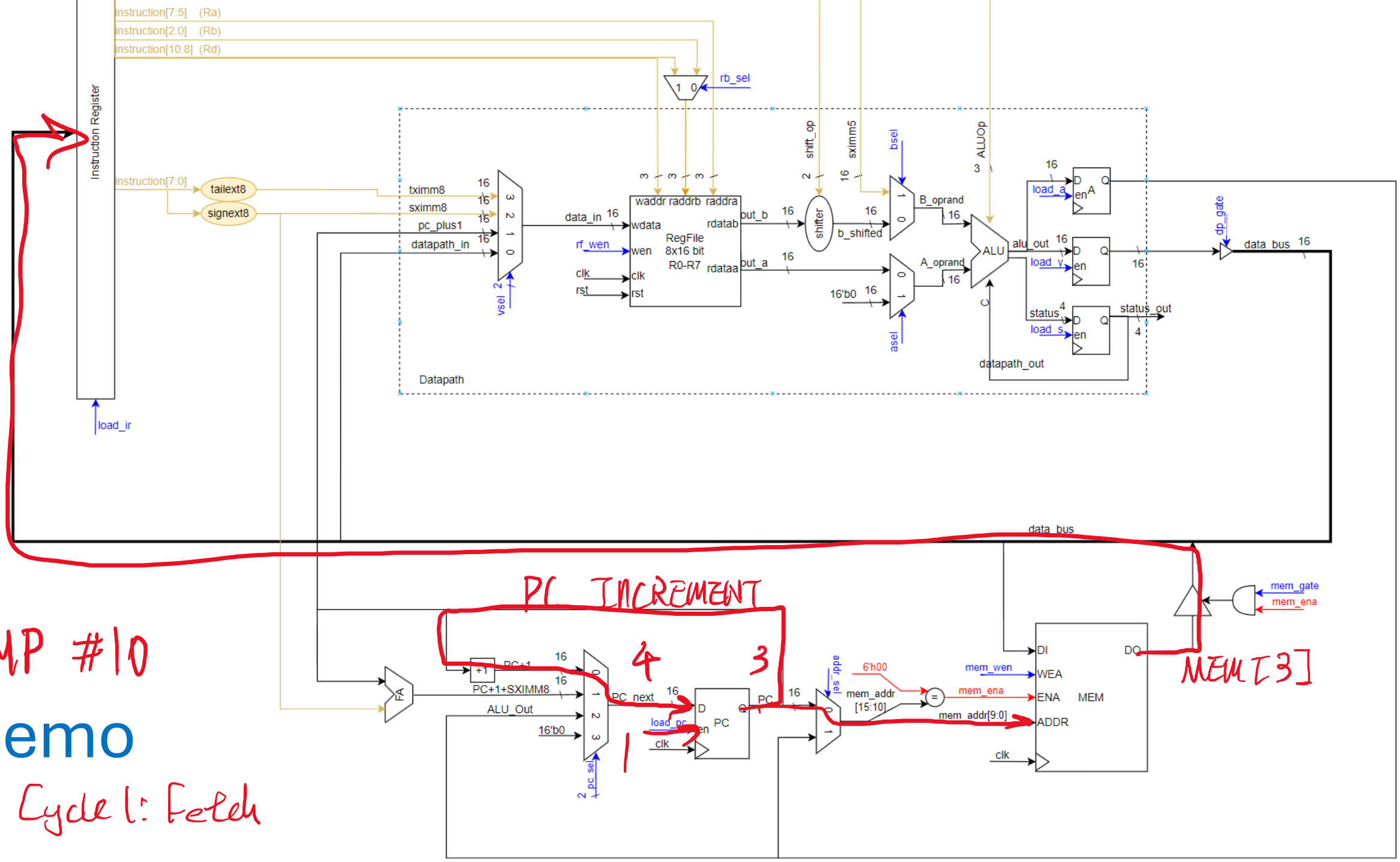


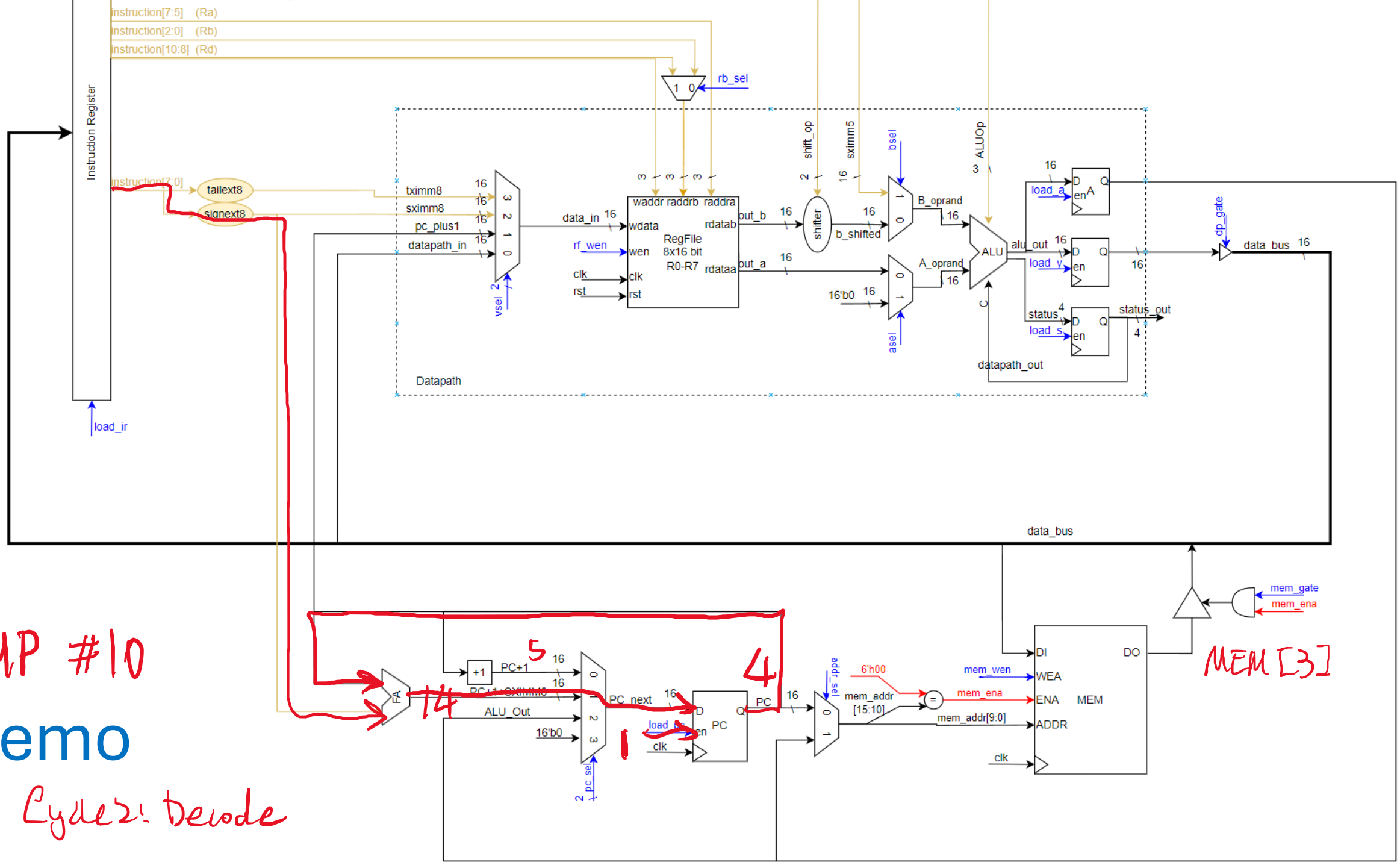
Cycle 4: STR MEM

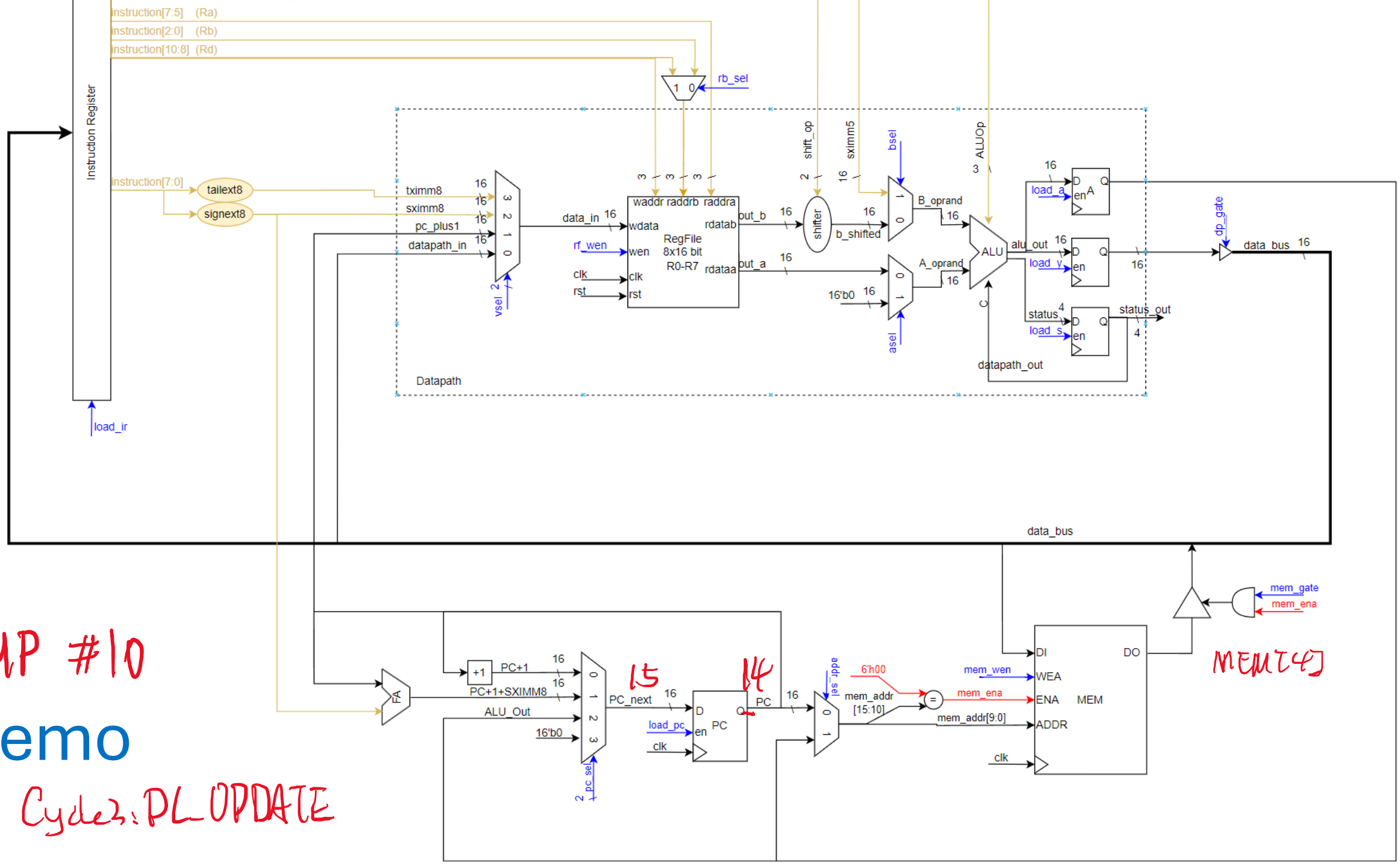
Demo


Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
instruction[7:0]
tailext8
signext8
load_ir
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
waddr raddrb raddra
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
shifter
sximm5
b_shifted
bsel
B_oprand
16'b0
asel
A_oprand
ALUOp
ALU
alu_out
load_a
load_y
status
load_s
status_out
datapath_out
Datapath
dp_gate
data_bus
mem_gate
mem_ena
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
mem_ena
DI
WEA
ENA
ADDR
DO
MEM
clk
4
3
Cycles: PC_UPDATE
MEM[Addr]

Demo


Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
instruction[7:0]
tailext8
signext8
load_ir
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
waddr raddr raddra
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
shifter
sximm5
b_shifted
bsel
B_oprand
16'b0
asel
A_oprand
ALUOp
ALU
alu_out
load_a
load_y
load_s
status
status_out
datapath_out
Datapath
dp_gate
data_bus
mem_gate
mem_ena
PC_INCREMENT
PC+1
PC+1+SXIMM8
ALU_Out
16'b0
PC_next
pc_sel
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_wen
mem_addr[9:0]
DI
WEA
ENA
ADDR
DO
MEM
JMP #10
Demo
Cycle 1: Fetch
MEM[3]
4
3
1


Instruction Register
instruction[7:5] (Ra)
instruction[2:0] (Rb)
instruction[10:8] (Rd)
rb_sel
instruction[7:0]
tailext8
signext8
load_ir
Datapath
tximm8
sximm8
pc_plus1
datapath_in
vsel
data_in
rf_wen
clk
rst
waddr raddr raddra
wdata
wen
RegFile
8x16 bit
R0-R7
rdatab
rdataa
out_b
out_a
shift_op
shifter
sximm5
b_shifted
bsel
B_oprand
16'b0
asel
A_oprand
ALUOp
ALU
alu_out
load_a
load_y
status
load_s
status_out
datapath_out
dp_gate
data_bus
mem_gate
mem_ena
PC+1
ALU_Out
16'b0
pc_sel
PC_next
load_pc
PC
addr_sel
6'h00
mem_addr
[15:10]
mem_addr[9:0]
mem_wen
DI
WEA
ENA
ADDR
MEM
DO
JMP #10
Demo
Cycle2: Decode
MEM[3]
5
4
14
1



JMP #10

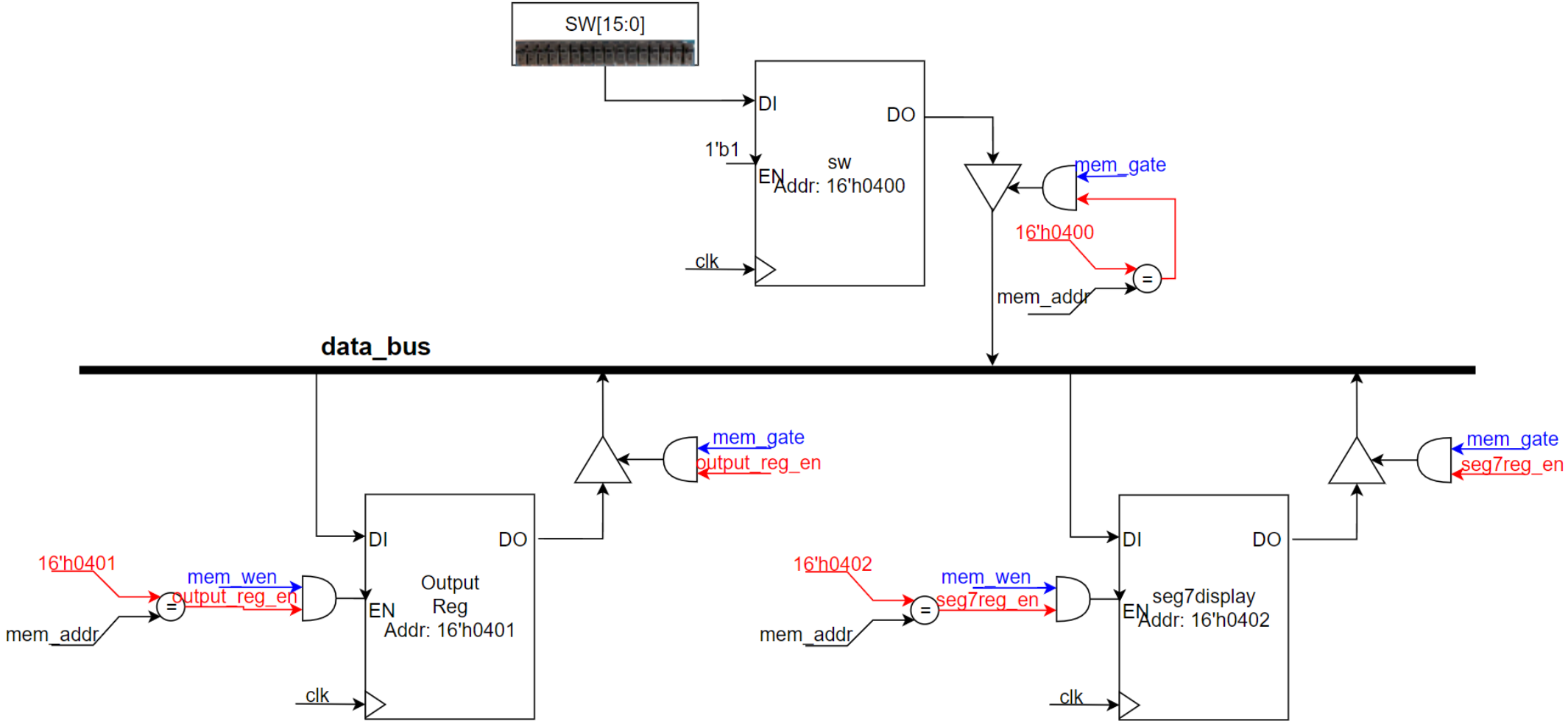
Demo

Cycle3: PL_UPDATE

IO

MEMORY-MAPPED PERIPHERALS

Memory-Mapped Peripherals: Special Function Registers



Memory Map for RISKing16

- Memory-mapping is a typical method to design peripherals. By giving the peripheral a memory address, the processor can deal with IO simply by reading and writing memory addresses like main memory.
- Example usage:

Address Rage	Description	RW
0xFFFF 0x0800	Unused Space	NA
0x07FF 0x0403	Reserved SFR space	NA
0x0402	7-segment display register	RW
0x0401	Output Register (LED)	RW
0x0400	Input Register	Read Only
0x03FF 0x0000	Main Memory	RW

```
; Load SFR base address (0x04 << 8 = 0x0400) into R0
LUI R0, 0x04            ; R0 = 0x0400 (SFR base address)

; Read input from switches
LDR R1, [R0, 0]         ; R1 = INPUT_REG (offset 0 from base)

; Write to LEDs
STR R2, [R0, 1]         ; OUTPUT_REG = R2 (offset 1 from base)

; Write to 7-segment display
STR R3, [R0, 2]         ; SEG7_REG = R3 (offset 2 from base)
```

R-type Instructions (Register Operations)

- example

```
ADD R1, R2, R3          ; R1 = R2 + R3 (no shift)
SUB R4, R5, R6          ; R4 = R5 - R6
ADD R0, R1, LSL(R2)     ; R0 = R1 + (R2 << 1)
ADD R0, R1, R2, LSL     ; R0 = R1 + (R2 << 1)  (alternative syntax)
AND R3, R4, ASR(R5)     ; R3 = R4 & (R5 >>> 1)
```

I-type Instructions (Register Operations)

- example

```
ADDI R1, R2, 10          ; R1 = R2 + 10
SUBI R3, R4, 5           ; R3 = R4 - 5
ADDI R5, R5, -1          ; R5 = R5 - 1 (decrement)
```

Load Immediate Instructions

- example

```
LDI R1, 100          ; R1 = 0x0064 (100, positive – bit 7 is 0)
LDI R2, 0xFF         ; R2 = 0xFFFF (-1, sign-extended – bit 7 is 1)
LUI R3, 0x04         ; R3 = 0x0400 (0x04 << 8)
LUI R4, 0x12         ; R4 = 0x1200 (0x12 << 8)
```

Memory Access Instructions

- example

```
LDR R3, [R1, 0]          ; R3 = Mem[R1 + 0]
LDR R4, R1, 5            ; R4 = Mem[R1 + 5] (alternative syntax)
STR R5, [R1, -2]         ; Mem[R1 - 2] = R5
```

Jump and Branch Instructions

- example

```
JMP loop_start          ; Unconditional jump
JAL subroutine          ; Call subroutine (return address in R7)
JR                      ; Return from subroutine (PC = R7)
BEQ skip                ; Branch if equal (Z flag set)
BNE continue            ; Branch if not equal (Z flag clear)
```

Special Instructions

- example

```
HALT                    ; Stop CPU execution
NOP                     ; No operation
MOV R1, R2              ; R1 = R2 (assembled as ADDI R1, R2, 0)
```

For complete instruction set:

RISC Processor Instruction Set Reference.xlsx
RISKing_Programming_Manual.md

alu.sv | RISKing_Programming_Manual.md RISKing_Stud_Proj\... | cpu_tb.sv RISKing16\... | cpu_tb.sv ...\src\

RISKing_Stud_Proj > docs > RISKing_Programming_Manual.md

RISKing CPU Programming Manual

Overview

> The RISKing processor is a 16-bit RISC CPU with a simple but powerful instruction set. This manual provides comprehensive guidance for writing assembly code for the RISKing CPU, including instruction syntax, programming conventions, memory layout, and special function registers.

Architecture Summary

- **Word Size**: 16 bits
- **Address Space**: 16-bit addresses (0x0000 - 0xFFFF)
- **Main Memory**: 1024 words (0x0000 - 0x03FF)
- **Registers**: 8 general-purpose registers (R0-R7)
- **Special Function Registers (SFR)**: 3 registers at addresses 0x0400-0x0402

Register Conventions

Register	Convention	Purpose
R0	General	Conventionally kept as zero (not hardware-enforced — software must initialize and preserve)
R1-R5	General	General-purpose registers
R6	**SP**	Stack Pointer (recommended usage)
R7	**LR**	Link Register (recommended usage)

> **Note**: While all registers are general-purpose, following these conventions ensures compatibility with standard calling conventions and makes code more readable.

Memory Layout

Assembly Program with RISKing: Syntax

Basic Structure
[label:] instruction [operands] ; comment
.DIRECTIVE [arguments]

Labels
End with colon (:)
Used for jump targets and data references
Case-sensitive

Comments
Single-line comments: ; or //
Comments are ignored by the assembler

Assembler Directives
.ORG address: Set the assembler's location counter to the specified address (decimal or hex)
.WORD value: Place a 16-bit data word at the current address (can be a number or a label)

Number Formats
Decimal: 10, -5
Hexadecimal: 0x10, 0xFF
Binary: Not directly supported (use hex)
Labels can be used as operands and are resolved to their address (relative for jumps/branches, absolute for .WORD)

Assembly Program with RISKing: Example

```
START:
    LDI R1, 10          ; R1 <- 0x000A (decimal: 10)
    LDI R2, 5           ; R2 <- 0x0005 (decimal: 5)
    ADD R3, R1, R2      ; R3 <- R1 + R2 = 0x000A + 0x0005 = 0x000F
    SUB R4, R1, R2      ; R4 <- R1 - R2 = 0x000A - 0x0005 = 0x0005
    AND R5, R1, R2      ; R5 <- R1 & R2 = 0x000A & 0x0005 = 0x0000
    OR  R6, R1, R2      ; R6 <- R1 | R2 = 0x000A | 0x0005 = 0x000F
    HALT                ; Stop CPU execution
```

; Register Allocation:
; R1 (Operand A) - First operand (value: 10)
; R2 (Operand B) - Second operand (value: 5)
; R3 (Result ADD) - Stores addition result
; R4 (Result SUB) - Stores subtraction result
; R5 (Result AND) - Stores bitwise AND result
; R6 (Result OR) - Stores bitwise OR result

; Expected Results:
; R3 = 15 (0x0F) - Addition: 10 + 5
; R4 = 5 (0x05) - Subtraction: 10 - 5
; R5 = 0 (0x00) - AND: 10 & 5 (binary: 1010 & 0101 = 0000)
; R6 = 15 (0x0F) - OR: 10 | 5 (binary: 1010 | 0101 = 1111)

Assembler: Convert Assembly to Binaries

- The RISKing assembler (RISKing_assembler.py) converts assembly code to binary format.
- Demo: Using RISKing_assembler
- Refer to **RISKing CPU Programming Manual.md**

```
START:
    LDI R1, 10
    LDI R2, 5
    ADD R3, R1, R2
    SUB R4, R1, R2
    AND R5, R1, R2
    OR  R6, R1, R2
    HALT
```

```
1000000100001010
1000001000000101
0000001100100010
0000110000100010
0010010100100010
0011011000100010
1111111111111111
```

Assembler: Convert Assembly to Binaries

Assembler Usage

The RISKing assembler (`RISKing_assembler.py`) converts assembly code to binary format.

Command Line

```
python RISKing_assembler.py input.asm [-o output_prefix] [-d memory_depth]
```

Parameters

- `input.asm`: Input assembly file
- `-o output_prefix`: Output file prefix (default: input filename)
- `-d memory_depth`: Memory depth in words (default: 1024)

Output Files

- `<prefix>.coe`: Vivado Block Memory initialization file (hex, COE format)
- `<prefix>.txt`: Binary format for simulation (`$readmemb`)
- `<prefix>.mem`: Memory initialization file (hex, one word per line)

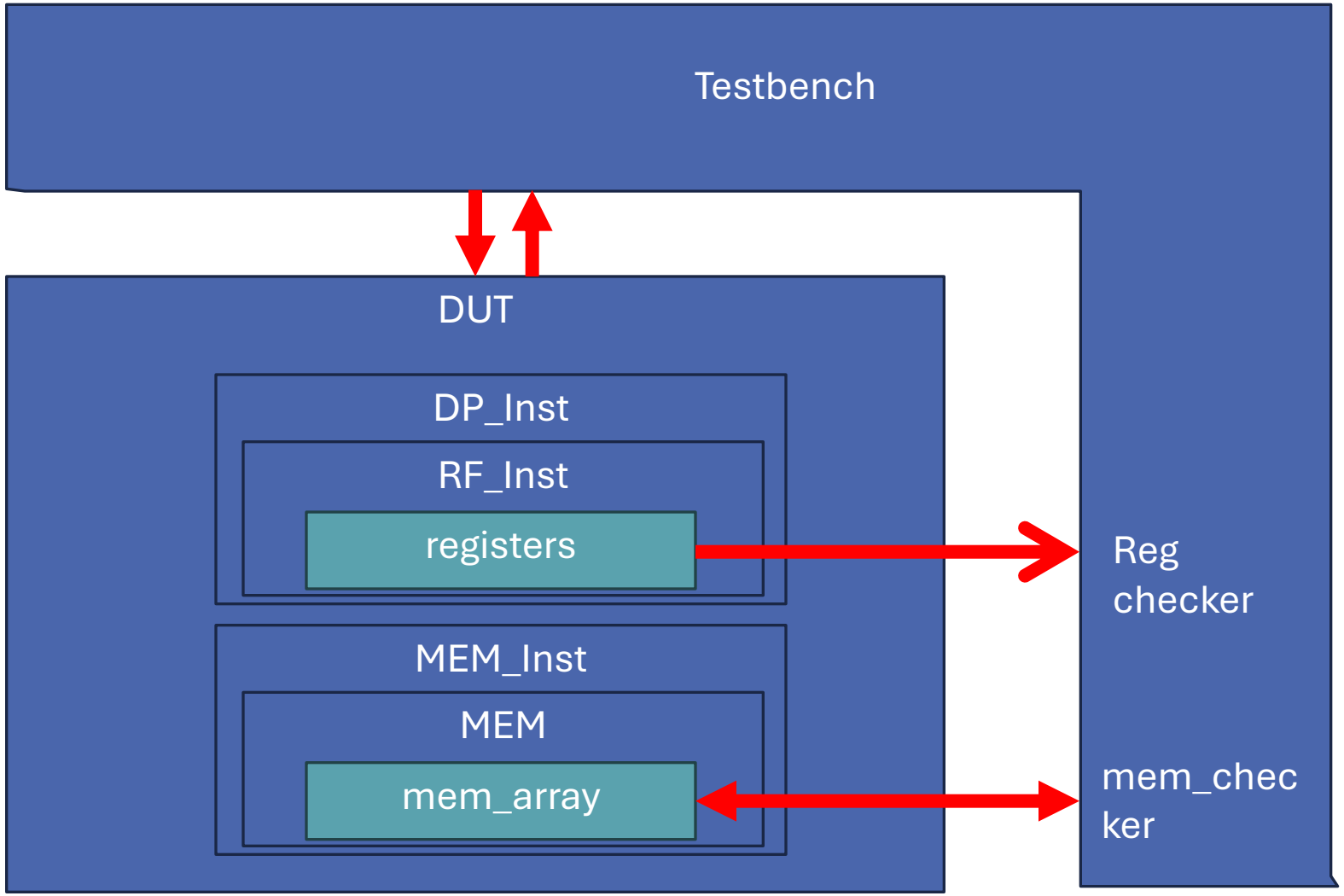
When `-o` is not specified, the output prefix defaults to `<input_basename>_mem_init` (e.g., `program.asm` → `program_mem_init.coe`, `program_mem_init.txt`, `program_mem_init.mem`).

Unused memory locations are filled with `0xFFFF` (HALT) up to the specified memory depth.

Example

```
python RISKing_assembler.py program.asm -o mem_init -d 1024
```

Code Evaluation: Testbench Hierarchy



Demo

- Finish CPU design
- Verify with Verilog Autograder With Waveforms
- Write your own assembly program
- Make a testbench with your program
- Download to BASYS3 for Demo